UỶ BAN NHÂN DÂN QUẬN 5 TPHCM

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO

ĐỀ TÀI:

TÌM HIỂU VÀ TRIỂN KHAI MẠNG SDN (Software Define Network)

Giảng viên hướng dẫn: Ths. Lương Minh Huấn

Nhóm sinh viên thực hiện:

3121410313 - Huỳnh Ngọc Diễm Ly

3120410454 - Trần Ngọc Sơn

3120410298 - Trịnh Cao Lộc

3121410054- Nguyễn Ngô Thế Anh

Thành phố Hồ Chí Minh, tháng 10 năm 2024

THÀNH VIÊN NHÓM			
STT	**HỌ TÊN**	**MSSV**	**Gmail**
1	Trần Ngọc Sơn	3120410454	shawkaito@gmail.com
2	Huỳnh Ngọc Diễm Ly	3121410313	diemly1603@gmail.com
3	Trịnh Cao Lộc	3120410298	caoloc061202@gmail.com
4	Nguyễn Ngô Thế Anh	3121410054	theanh1148@gmail.com

LỜI CẢM ƠN

Lời đầu tiên chúng em xin chân thành cảm ơn các thầy cô trong khoa Công nghệ thông tin của trường đại học Sài Gòn. Trong suốt hai năm qua, sự hướng dẫn và sự chia sẻ kiến thức từ các thầy cô đã làm cho chúng em trang bị được những kiến thức và kỹ năng cơ bản, là nền tảng quý báu cho tương lai sự nghiệp của chúng em. Chúng em không thể không đặc biệt bày tỏ lòng biết ơn thầy Lương Minh Huấn người đã dành thời gian và tận tâm hướng dẫn nhóm của chúng em trong việc hoàn thành dự án ngành. Sự quan tâm và sự hỗ trợ từ các thầy đã giúp chúng em tiến bộ và tự tin hơn trong quá trình nghiên cứu và thực hiện dự án.

Chúng em cũng muốn bày tỏ lòng biết ơn sâu sắc đến gia đình và bạn bè, những người đã luôn ủng hộ và chia sẻ cảm xúc trong suốt thời gian chúng em học tập và nghiên cứu. Đặc biệt, chúng em muốn kính mến các thành viên trong nhóm, những người đã cùng nhau làm việc chăm chỉ và đóng góp ý kiến quý báu để chúng em hoàn thành dự án này một cách thành công. Trong quá trình hoàn thành dự án, chúng em cũng đối mặt với nhiều khó khăn và thách thức, nhưng qua thời gian học tập và nghiên cứu chúng em cũng khắc phục được nhiều yếu điểm. Sự thiếu kinh nghiệm thực tế và kiến thức đã khiến cho chúng em gặp phải những sai sót và hạn chế. Chúng em xin trân trọng mọi sự góp ý và nhận xét từ các thầy cô để chúng em có thể học hỏi và phát triển kiến thức của mình, áp dụng chúng vào thực tế một cách hiệu quả trong tương lai.

Nhóm chúng em xin chân thành cảm ơn!

Các tiêu chuẩn mạng máy tính đã và đang trải qua một giai đoạn phát triển vượt bậc trong suốt 2 thập kỷ gần đây. Sự phức tạp trong việc tích hợp các giải pháp bảo mật cho các luồng dữ liệu khi cần chuyển đổi công năng trong hệ thống Network, phục vụ cho các mục đích khác nhau trong hệ thống. Các quyết định xử lý như thế nào đối với từng luồng lưu lượng hiện tại đang được thực hiện trên các thiết bị riêng biệt như switch/router… đó là một vài vấn đề tồn tại trong các hệ thống mạng. Công nghệ SDN - Software defined Networking (Mạng định nghĩa bằng phần mềm) ra đời như một giải pháp cho hệ thống Network hiện nay, đồng thời nó cũng đặt ra những thách thức trong việc đảm bảo an ninh và an toàn dữ liệu trong thời gian tới.

SDN là một trong những ứng dụng phổ biến đối với mạng truyền tải hiện nay mà các nhà cung cấp dịch vụ viễn thông định hướng triển khai, với việc phân tách phần điều khiển mạng (Control Plane) và chức năng vận chuyển dữ liệu (Forwarding Plane/Data Plane), SDN cho phép việc điều khiển mạng có thể lập trình được dễ dàng và cơ sở hạ tầng mạng độc lập với các ứng dụng và dịch vụ mạng.

Đối tượng nghiên cứu: *Công nghệ SDN; Kiến trúc và cách thức hoạt động của SDN; Kiến trúc và giao thức OpenFlow.*

Phương pháp nghiên cứu:

- Nghiên cứu về lý thuyết về SDN

- Nghiên cứu về lý thuyết về Openflow

- Tìm hiểu về các nội dung liên quan: Mininet, các giao thức và ứng dụng trong hệ

Phạm vi nghiên cứu:

- Tổng quan về SDN (Software Defined Network), OpenFlow

- *Triển khai thử nghiệm:* Thiết kế hệ thống, xây dựng kịch bản thử nghiệm, tổ chức tích hợp, cấu hình hệ thống.

Báo cáo sẽ bao gồm các chương sau:

Chương I: Tổng quan về Software Defined Networking – SDN

Chương II: Tổng quan về OpenFlow

Chương III: Các vấn đề an toàn trong SDN

Chương IV: Các bộ điều khiển – SDN Controller phổ biến

Chương V: Phương án thiết kế

Chương VI: Demo

Chương VII: Hướng phát triển

MỤC LỤC

CHƯƠNG I: TỔNG QUAN VỀ SOFTWARE DEFINED NETWORK - SDN 13

I. Giới thiệu 13

1. Sự ra đời của SDN **13**
2. Khái niệm về SDN **16**

II. Sự khác biệt giữ SDN với mạng truyền thống 18

1. Kiến trúc **18**
2. Tính năng **19**

III. Kiến trúc của SDN 21

1. Lớp ứng dụng - Application Layer **21**
2. Lớp hạ tầng - Infrastructure layer **21**
3. Lớp điều khiển - Control Layer **21**
4. Giới thiệu về API trong SDN **21**

IV. Cách hoạt động của SDN 22

V. Lợi ích và thách thức của SDN 23

1. Lợi ích của SDN **23**
2. Thách thức của SDN **25**

CHƯƠNG II: TỔNG QUAN VỀ OPENFLOW 27

I. Giới thiệu về Openflow 27

II. Lịch sử phát triển của OpenFlow: 28

III. Các đặc trưng của OpenFlow: 28

IV. Kiến trúc và hoạt động của OpenFlow 29

1. Kiến trúc của OpenFlow **29**
2. Hoạt động của OpenFlow **30**

V. Ưu điểm của OpenFlow 32

CHƯƠNG III: CÁC VẤN ĐỀ AN TOÀN TRONG SDN 34

I. Các mối đe dọa và giải pháp 34

1. Giả mạo luồng traffic **34**
2. Khai thác lỗ hổng trong switch **34**
3. Khai thác lỗ hổng trong bộ điều khiển **34**

II. Triển khai mạng an toàn trong SDN 35

1. Triển khai - chuyển đổi từng bước thích hợp **35**

2. Giải quyết những khó khăn, quan ngại **36**

CHƯƠNG IV: CÁC BỘ ĐIỀU KHIỂN - SDN CONTROLLER PHỔ BIẾN 37

I. Các bộ điều khiển trong SDN 37

1. OpenDayLight 37
2. Open Network Operating System (ONOS) 38
3. RYU 39
4. POX 39

II. Mininet 40

1. Định nghĩa 40
2. Tính năng 40

CHƯƠNG V: HƯỚNG PHÁT TRIỂN 42

I. Đề xuất phương án: 42

II. Lựa chọn phương án: 42

CHƯƠNG VI: DEMO 44

I. Mục tiêu: 44

II. Các yêu cầu cài đặt và công cụ: 44

III. Quá trình thực hiện đo iperf3 khi bị tấn công bằng hping3 44

CHƯƠNG VI: HƯỚNG PHÁT TRIỂN 47

TÀI LIỆU THAM KHẢO 51

TÀI LIỆU HƯỚNG DẪN CÀI ĐẶT 53

DANH MỤC HÌNH

Hình 1 Kiến trúc SDN 15

Hình 2 Kiến trúc mạng truyền thống 16

Hình 3 Kiến trúc mạng SDN 16

Hình 4 Minh họa giao thức OpenFlow 27

Hình 5 Bảng lưu lượng trong OpenFlow 28

Hình 6 Cấu trúc Openflow Switch 29

Hình 7 Ví dụ tạo topo mạng theo mong muốn 43

Hình 8 Hình ảnh trước khi chưa custom name cho thiết bị được 46

Hình 9 Chạy iperf3 Server trên Host 9 49

Hình 10 Host 9 trước khi bị tấn công 49

Hình 11 Thực hiện tấn công lên Host 9 49

Hình 12 Kết quả sau khi bị tấn công 50

Hình 13 Kết quả sau khi ngừng tấn công 50

DANH MỤC BẢNG

Bảng 1. Từ viết tắc và nguyên nghĩa 6

Bảng 2. So sánh sự khác biệt giữa mạng truyền thống và và mạng SDN 18

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Từ gốc	Ý nghĩa
1	SDN	Sofware Defined Network	Mạng sử dụng phần mềm để quản lý và điều phối tài nguyên mạng.
2	BYOD	Bring Your Own Device	Chính sách cho phép nhân viên sử dụng thiết bị cá nhân trong công việc.
3	DC	Data center	Cơ sở hạ tầng tập trung cho việc lưu trữ, xử lý và phân phối dữ liệu.
4	VMs	Virtual Machines	Các môi trường ảo hóa cho phép chạy nhiều hệ điều hành trên một phần cứng – Máy ảo.
5	QoS	Quality of service	Kỹ thuật đảm bảo hiệu suất và độ tin cậy trong việc truyền tải dữ liệu.
6	ACL	Access List	Danh sách quy định quyền truy cập cho người dùng hoặc thiết bị.
7	ONF	Open Networking Foundation	Tổ chức phi lợi nhuận thúc đẩy tiêu chuẩn mở cho mạng lưới.
8	API	Application Programming Interface	Giao thức cho phép các phần mềm giao tiếp và tương tác với nhau.

9	NVF	Network Function Virtualization	Kỹ thuật ảo hóa các chức năng mạng để giảm chi phí và tăng tính linh hoạt.
10	IoT	Internet of Things	Mạng lưới thiết bị vật lý kết nối internet và có khả năng thu thập dữ liệu.
11	DoS	Denied of Service	Tấn công nhằm làm gián đoạn dịch vụ bằng cách làm quá tải tài nguyên.
12	ODL	Open Day Light	Nền tảng điều khiển mạng mã nguồn mở cho các giải pháp SDN.

Bảng 1 Từ viết tắc và nguyên nghĩa

NHẬN XÉT CỦA GIẢNG VIÊN

TP Hồ Chí Minh, ngày…tháng…năm 2024

Giảng viên hướng dẫn

Thầy Lương Minh Huấn

CHƯƠNG I: TỔNG QUAN VỀ SOFTWARE DEFINED NETWORK - SDN

I. Giới thiệu

1. Sự ra đời của SDN

Sự bùng nổ của số lượng thiết bị kết nối mạng, gồm cả thiết bị di động lẫn IoT, kéo theo đó là sự bùng nổ về nội dung, dữ liệu được truyền tải trên mạng, đặt ra một áp lực rất lớn đối với mạng truyền thống, vốn dĩ được thiết kế với khả năng mở rộng (scale-out) nhất định.

Tiếp đến, hạ tầng tính toán và lưu trữ (computing and storage) đã có những phát triển vượt bậc trong những năm gần đây. Do đó, hạ tầng mạng không chỉ thuần túy kết nối theo kiểu truyền thống Client-Server mà còn phải đảm bảo năng lực kết nối cho hạ tầng computing và storage này. Đặc thù của computing và storage ngày nay là khả năng mở rộng, thu hẹp cực kỳ nhanh chóng và linh hoạt. Điều này đặt ra yêu cầu lớn cho độ linh hoạt của hạ tầng mạng.

Sự phát triển của ảo hóa dẫn đến traffic trong mạng cũng thay đổi rất nhiều, nhất là trong các Data center. Ngày nay, traffic từ người dùng đến server (North-south traffic) thấp hơn hơn rất nhiều so với traffic từ server đến server (East-West traffic). Mô hình mạng phân cấp ba lớp (Core/Distribution/Access) đã từng rất hiệu quả trong kiến trúc mạng Client-Server. Tuy nhiên, khi khối lượng East-West traffic tăng lên đột biến thì kiến trúc mạng phân cấp (hierarchical) này trở nên cứng nhắc, nặng nề và kém hiệu quả, bắt buộc phải có một kiến trúc mới hơn đáp ứng xu thế này.

BYOD (Bring your own device) trở thành xu hướng mới của doanh nghiệp, theo đó người dùng trong doanh nghiệp được phép sử dụng các thiết bị cá nhân, và thậm chí nhiều thiết bị cùng lúc để truy xuất dữ liệu và làm việc hằng ngày, chứ không chỉ những thiết bị của doanh nghiệp với những chính sách truy cập ràng buộc. Điều này đặt ra thách thức lớn cho đội ngũ quản trị, vận hành IT, nhất là luồng traffic truy cập linh hoạt và thay đổi khá nhiều so với sự tập trung trong kiến trúc mạng truyền thống trước đây.

Multi Data Center (DC) cũng là điểm đáng chú ý. Các doanh nghiệp không chỉ xây dựng một DC mà còn nhiều DC với nhu cầu phục hồi thảm họa (DR – Disater recovery) hoặc chia tải. Các tác vụ như Data Replication hay Virtual Machine mobility giữa các DCs lại một lần nữa đẩy nhu cầu East-West traffic lên cao. Việc sử dụng các kiến trúc, mô hình, công nghệ truyền thống để thiết kế cho các kết nối giữa các Data center ngày nay trở nên phức tạp và rất khó đáp ứng.

Các dịch vụ cloud vẫn đang rất phát triển. Khả năng built-up nhanh chóng, quản trị đơn giản, co giãn tài nguyên linh hoạt theo yêu cầu… là những ưu điểm đáng chú ý khiến các doanh nghiệp đón nhận mạnh mẽ các dịch vụ "trên mây" này. Doanh nghiệp thường có cả hạ tầng DC riêng (private cloud) và hạ tầng trên public cloud. Khả năng tích hợp, tối ưu giữa kiến trúc hybrid như vậy cũng đặt ra rất nhiều thách thức mới cho hạ tầng mạng nói riêng và hạ tầng CNTT nói chung của doanh nghiệp.

Trong bối cảnh cạnh tranh giữa các tổ chức. Khả năng cung cấp (provisioning) nhanh chóng hạ tầng CNTT để triển khai dịch vụ trở thành công cụ cạnh tranh rất quan trọng. Các doanh nghiệp cần đẩy nhanh các dự án xây dựng hạ tầng CNTT cũng như khả năng provisoning các ứng dụng, dịch vụ mạng chỉ trong vài ngày, thậm chí vài giờ thay vì phải kéo dài hàng tháng như trước đây, nhằm nâng cao tính cạnh tranh và tập trung nguồn lực vào các chiến lược kinh doanh khác. Ngoài ra, ngày nay các công ty có xu hướng sáp nhập, mua lại, dẫn tới nhu cầu hạ tầng CNTT cũng cần có tính mở, có khả năng sáp nhập, tích hợp cao. Điều đó cũng góp phần cho nhu cầu xây dựng một hệ thống mạng đáp ứng những tiêu chuẩn chung, nâng cao tính khả mở và tự động trong tương lai.

Như vậy, ngày nay, networking "kết nối cái gì" và "kết nối như thế nào" đã có nhiều thay đổi. Hàng tetrabyte lưu lượng traffic hoạt động trên mạng.

Cách thức traffic xuất hiện và di chuyển cũng đã khác hẳn so với trước đây. Khả năng triển khai nhanh chóng, mở rộng linh hoạt, tăng cường tự động hóa, tích hợp dễ dàng trở thành những thuộc tính quan trọng mà bắt buộc hệ thống mạng ngày nay phải đáp ứng.

Trong khi đó, hệ thống mạng truyền thống hiện tại bộc lộ rất nhiều điểm hạn chế. Hệ thống quá phức tạp dẫn đến cứng nhắc, không linh hoạt để đáp ứng các thay đổi theo nhu cầu kinh doanh của doanh nghiệp. Đối với từng lĩnh vực hoặc yêu cầu khác nhau của doanh nghiệp, networking lại có những thiết kế riêng biệt với quá nhiều các công nghệ và giao thức rời rạc khác nhau, lại có thể phụ thuộc từng hãng công nghệ riêng biệt, mà không có một cơ chế nào giúp trừu tượng hóa (abstraction) hoặc có khả năng vận hành tập trung và nhất quán. Một ví dụ để thấy sự cứng nhắc của hệ thống mạng hiện tại, khi mà các server (VMs) dễ dàng di chuyển tự động từ chỗ này sang chỗ khác dựa trên sự hỗ trợ của ảo hóa, thì các chính sách mạng (định tuyến, ACL, QoS…) lại phải thực hiện di chuyển thủ công. Một tác vụ VM migration phổ biến của ảo hóa lại trở nên thách thức cho hạ tầng mạng, khi phải giải quyết các vấn đề liên quan IP, default gateway, thậm chí, cấu trúc định tuyến cũng phải thay đổi.

Nói về khả năng mở rộng, lại là một hạn chế lớn đối với networking truyền thống. Khi thiết kế, các nhà thiết kế mạng có thể tính toán để dự phòng cho sự phát triển của doanh nghiệp trong vòng 3 năm, 5 năm… nhưng đó vẫn là con số cố định. Trong khi chính các doanh nghiệp cũng khó có thể lường trước được khả năng mở rộng kinh doanh của họ, và mở rộng hạ tầng CNTT tương ứng, nhất là các doanh nghiệp phát triển, cung cấp dịch vụ cloud. Chưa kể, nhu cầu mở rộng hạ tầng của doanh nghiệp có thể nhất thời. Ví dụ, doanh nghiệp yêu cầu tài nguyên mạng tăng lên gấp 2 lần để phục vụ business kéo dài 1 tháng và trả lại như cũ ngay sau đó. Đứng trước yêu cầu mở rộng hạ tầng thường xuyên, nhanh chóng và dễ dàng của các doanh nghiệp thì kiến trúc mạng, thiết kế mạng ngày nay gần như không đáp ứng kịp.

Khi hạ tầng mạng phát triển và mở rộng, đồng nghĩa số lượng thiết bị (node mạng) tăng lên, số lượng kết nối, đường truyền tăng lên, các chính sách truy cập, định tuyến, QoS cũng trở nên dày đặc. Nhưng tất cả các cấu hình ấy, người quản trị phải thực hiện thủ công, trên từng thiết bị. Một hệ thống phức tạp lại không có khả năng tự động hóa dẫn đến đội ngũ vận hành gặp quá tải trong

các tác vụ triển khai, vận hành cũng như xuất hiện nhiều sai sót chủ quan từ người quản trị.

Như vậy, đứng trước những thay đổi lớn lao từ các các lĩnh vực liên quan trong ngành công nghệ, và những hạn chế trong kiến trúc mạng truyền thống vốn đã tồn tại hàng thập niên qua; chúng ta cần có một cách tiếp cận mới hơn, một kiến trúc mạng mới hơn nhằm đáp ứng với xu thế chung của áo hóa, cloud, và những yêu cầu từ ứng dụng của doanh nghiệp nói chung. Và, những yêu cầu ấy cũng chính là cảm hứng để ra đời một xu thế mới trong ngành công nghiệp mạng: Software Defined Network (SDN)

2. Khái niệm về SDN

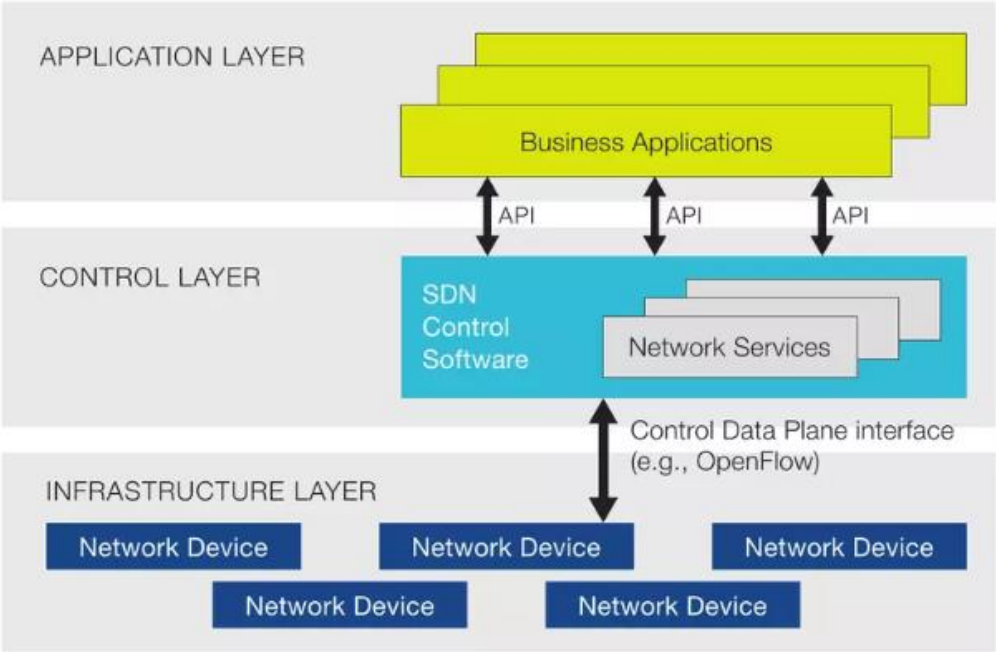
Hiện nay có rất nhiều định nghĩa về mạng SDN - Software Defined Network nhưng theo ONF (Open Networking Foundation – một tổ chức phi lợi nhuận đang hỗ trợ việc phát triển SDN thông qua việc nghiên cứu các tiêu chuẩn mở phù hợp) thì mạng SDN được định nghĩa như sau: "Mạng định nghĩa được bằng phần mềm hay Sofware Defined Network (SDN) là một kiểu kiến trúc mạng mới, năng động, dễ quản lý, chi phí hiệu quả, dễ thích nghi và rất phù hợp với nhu cầu mạng ngày càng tăng hiện nay. Kiến trúc này phân tách phần điều khiển mạng (Control Plane) và chức năng vận chuyển dữ liệu (Forwarding Plane hay Data Plane), điều này cho phép việc điều khiển mạng trở nên có thể lập trình được dễ dàng và cơ sở hạ tầng mạng độc lập với các ứng dụng và dịch vụ mạng".

Công nghệ mạng được xác định bằng phần mềm (Software-defined network – SDN) là một cách tiếp cận theo hướng điện toán đám mây, tức là tập trung hóa việc quản trị thiết bị mạng, giảm thiểu công việc riêng lẻ, tốn nhiều thời gian trên từng thiết bị cụ thể, tạo điều kiện thuận lợi cho việc quản lý mạng và cho phép cấu hình mạng hiệu quả bằng các chương trình được lập trình để cải thiện hiệu suất và tăng cường khả năng giám sát mạng. SDN nhằm mục tiêu giải quyết vấn đề kiến trúc tĩnh, phi tập trung, phức tạp của các mạng truyền thống không phù hợp với yêu cầu về tính linh hoạt và xử lý sự cố dễ dàng của các hệ thống mạng hiện tại.

SDN dựa trên giao thức luồng mở (Open Flow) và là kết quả nghiên cứu của Đại học Stanford và California Berkeley. SDN tách định tuyến và chuyển các luồng dữ liệu riêng rẽ và chuyển kiểm soát luồng sang thành phần mạng riêng có tên gọi là thiết bị kiểm soát luồng (Flow Controller). Điều này cho phép luồng các gói dữ liệu đi qua mạng được kiểm soát theo lập trình.

Trong SDN, control plane được tách ra từ các thiết bị vật lý và chuyển đến các bộ điều khiển. Bộ điều khiển này có thể nhìn thấy toàn bộ mạng lưới, và do đó cho phép các kỹ sư mạng làm cho chính sách chuyển tiếp tối ưu dựa trên toàn bộ mạng. Các bộ điều khiển tương tác với các thiết bị mạng vật lý thông qua một giao thức chuẩn OpenFlow.

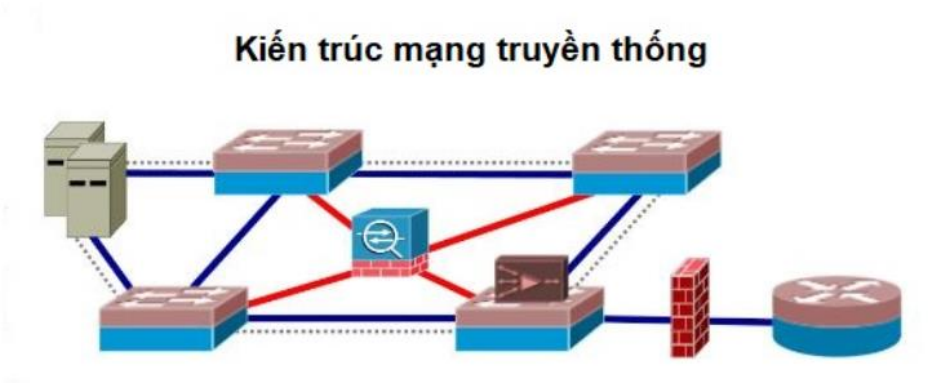
Kiến trúc của SDN gồm 3 lớp riêng biệt: lớp ứng dụng, lớp điều khiển, và lớp cơ sở hạ tầng (lớp chuyển tiếp).



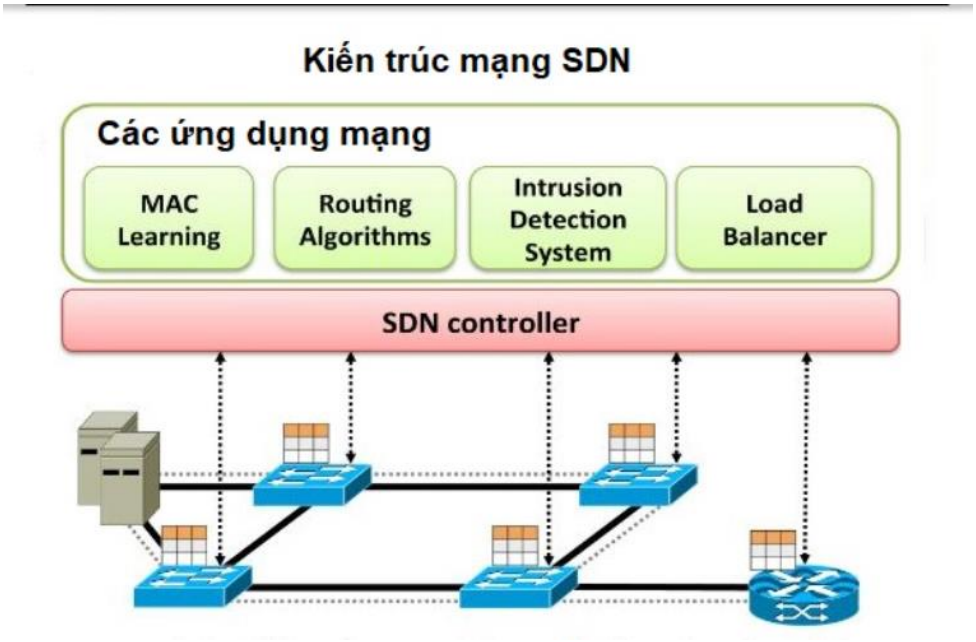
Hình 1 Kiến trúc SDN

II. Sự khác biệt giữ SDN với mạng truyền thống

1. Kiến trúc



Hình 2 Kiến trúc mạng truyền thống



Hình 3 Kiến trúc mạng SDN

SDN cố gắng tập trung việc quản trị mạng vào một thành phần trong mạng bằng cách tách rời quá trình chuyển tiếp của các gói tin trong mạng (mặt phẳng dữ liệu) khỏi quá trình định tuyến (mặt phẳng điều khiển). Mặt phẳng điều khiển (SDN controller) bao gồm một hoặc nhiều bộ điều khiển được coi là bộ não của mạng SDN nơi kết hợp toàn bộ trí thông minh hay các thuật toán định tuyến, bảo mật phức tạp của mạng.

Như ở hình trên ta thấy, các thiết bị mạng chỉ còn phần chức năng chuyển tiếp dữ liệu màu xanh, còn phần màu nâu (phần định tuyến) đã được chuyển dịch sang một thiết bị khác đóng vai trò điều khiển cho toàn mạng gọi là SDN controller. Trên đó chứa toàn bộ các thuật toán định tuyến, các giải pháp an toàn, bảo mật và cân bằng tải. Khi cấu hình cho toàn mạng, quản trị viên chỉ cần viết và thay đổi các thuật toán trên thiết bị này bằng phần mềm, toàn bộ các thiết mạng sẽ được quản trị đồng bộ. Khi muốn thay đổi bất kỳ thông số hay yêu cầu gì cho mạng, chỉ việc thay đổi trên phần mềm quản lý của SDN controller. Điều này hoàn toàn khác so với công nghệ mạng truyền thống ở trên khi phần màu xanh (điều khiển) và phần màu nâu (phần định tuyến) nằm trên cùng một thiết bị và do đó, khi muốn thay đổi cấu hình cho mạng, các quản trị viên buộc phải thay đổi thủ công trên từng thiết bị.

SDN là một tiêu chuẩn mới nổi mô tả các thay đổi trong trạng thái hiện tại của mạng máy tính, bằng cách tách phần logic điều khiển mạng khỏi các bộ chuyển mạch và bộ định tuyến cơ bản, thúc đẩy điều khiển mạng tập trung và cung cấp khả năng lập trình cho mạng. Với SDN, chúng ta có thể dễ dàng tạo và triển khai các quy tắc, cấu hình mới trong mạng, đơn giản hóa việc quản lý mạng và tạo điều kiện cho sự phát triển của mạng máy tính. Mạng SDN cho thấy sự khác biệt và cải tiến so với các công nghệ mạng máy tính truyền thống cũng như những lợi ích của các mạng SDN được triển khai với sự trợ giúp của công nghệ ảo hóa mạng. Giờ đây các thiết bị mạng như router hay switch chỉ đóng vai trò chuyển tiếp gói tin và không cần nắm giữ các nhiệm vụ điều khiển phức tạp nữa, toàn bộ phần mềm điều khiển này có thể được chuyển lên một thiết bị controller chuyên biệt.

2. Tính năng

Một thiết bị mạng như router hay switch bình thường bao gồm 2 phần đó là phần cứng và phần mềm. Phần mềm đảm bảo chức năng trao đổi các thông tin với các thiết bị router/switch khác và tính toán các đường định tuyến dựa trên các thông tin đã thu thập được. Phần cứng đảm nhiệm chức năng chuyển tiếp các gói tin đến theo một lộ trình đã được tính toán sẵn. Đối với mạng truyền thống

thì các thiết bị định tuyến hoặc chuyển mạch trao đổi các thông tin với nhau và quá trình tính toán xử lý đều xảy ra ở mỗi node mạng (ở tại mỗi router/switch). Nhưng đối với mạng SDN thì điều đó lại khác, với SDN việc thu thập thông tin của các thiết bị trong mạng và tính toán xử lý các thông tin thu thập được đều được chuyển đến một bộ điều khiển mạng (Network Controller). Các thiết bị router/switch chỉ tập trung vào chức năng vận chuyển dữ liệu. Điều đó làm cho việc quản lý mạng trở nên đơn giản hơn và các thiết bị phần cứng có thể nâng công suất làm việc lên.

Sự khác biệt cơ bản giữa mạng truyền thống và mạng SDN

Tiêu chí	**Mạng truyền thống**	**Mạng SDN**
Phần điều khiển	Tích hợp trong thiết bị mạng	Tách riêng khỏi thiết bị mạng và được thực hiện bởi bộ điều khiển SDN (SDN Controller)
Phần vận chuyển dữ liệu	Tích hợp trong thiết bị mạng	Thực hiện bởi các thiết bị mạng, nhưng được điều khiển bởi bộ điều khiển SDN
Thu thập và xử lý thông tin	Được thực hiện ở tất cả các phần tử trong mạng	Tập trung xử lý tại bộ điều khiển SDN
Khả năng lập trình	Không thể lập trình bởi các ứng dụng	Có thể lập trình bởi các ứng dụng, thông qua bộ điều khiển SDN
Cấu hình thiết bị mạng	Phải cấu hình từng thiết bị một cách thủ công và riêng lẻ	Bộ điều khiển SDN có thể cấu hình và điều khiển toàn bộ thiết bị trong mạng

Bảng 2 So sánh sự khác biệt giữa mạng truyền thống và và mạng SDN

III. Kiến trúc của SDN

1. Lớp ứng dụng - Application Layer

Lớp ứng dụng, đúng như tên gọi của nó, chứa các ứng dụng hoặc các chức năng mạng điển hình mà các tổ chức sử dụng như các hệ thống phát hiện xâm nhập, cân bằng tải hoặc tường lửa.

Điều này khác với một mạng truyền thống, với kiến trúc mạng truyền thống, trong mạng truyền thống, để thực hiện các chức năng này, mạng sẽ cần sử dụng một thiết bị chuyên dụng, như tường lửa hoặc thiết bị cân bằng tải riêng biệt. Trong khi với SDN các thiết bị này sẽ được thay thế bằng một ứng dụng phần mềm sử dụng bộ điều khiển để quản lý hành vi của mặt phẳng dữ liệu.

2. Lớp hạ tầng - Infrastructure layer

Lớp hạ tầng bao gồm các thiết bị chuyển mạch vật lý trong mạng

3. Lớp điều khiển - Control Layer

Lớp điều khiển đại diện cho phần mềm điều khiển SDN - SDN Controller tập trung hoạt động như bộ não của mạng. Bộ điều khiển này nằm trên một máy chủ và quản lý các chính sách và luồng lưu lượng trên toàn mạng.

4. Giới thiệu về API trong SDN

API (Application Programming Interface) là một tập hợp các giao thức, quy tắc và công cụ cho phép các phần mềm hoặc hệ thống giao tiếp và tương tác với nhau. Nói cách khác, API cung cấp cách thức mà một chương trình có thể yêu cầu thông tin hoặc thực hiện các chức năng từ một chương trình khác mà không cần phải hiểu chi tiết cách chương trình đó hoạt động nội bộ.

Trong **SDN**, API giúp các thành phần như lớp ứng dụng, lớp điều khiển, và lớp hạ tầng giao tiếp với nhau một cách hiệu quả:

+ **Southbound API** là cầu nối giữa lớp điều khiển và lớp hạ tầng. Các giao thức Southbound API cho phép bộ điều khiển SDN giao tiếp với các thiết bị mạng (như bộ chuyển mạch). Một giao thức Southbound API phổ biến nhất là OpenFlow, được xem như tiêu chuẩn đầu tiên và được sử dụng rộng rãi nhờ vào

lịch sử phát triển của nó. Mặc dù có nhiều giao thức khác tồn tại, OpenFlow vẫn là sự lựa chọn phổ biến trong nhiều triển khai SDN.

+ **Northbound API** là cầu nối giữa lớp ứng dụng và lớp điều khiển trong kiến trúc SDN. Các phần mềm phía tầng ứng dụng giao tiếp với lớp điều khiển thông qua Northbound API. Hiện tại, chưa có một tiêu chuẩn chung cho Northbound API, nhưng OpenDaylight nổi lên như một giải pháp thực tế và được hỗ trợ bởi nhiều nhà cung cấp.

Khi nhắc đến SDN, OpenFlow thường được đề cập đến nhiều nhất vì nó là giao thức Southbound API đầu tiên. OpenFlow cho phép bộ điều khiển SDN quản lý và điều khiển luồng dữ liệu qua các thiết bị mạng. Mặc dù hiện tại có nhiều giao thức Southbound khác, OpenFlow vẫn giữ vị trí quan trọng và phổ biến trong nhiều môi trường SDN.

IV. Cách hoạt động của SDN

Ý tưởng của SDN dựa trên sự kết hợp của việc tách biệt các chức năng điều khiển và chuyển tiếp dữ liệu, ảo hóa mạng và tự động hóa thông qua khả năng lập trình.

Ban đầu, công nghệ SDN chỉ tập trung vào việc tách mặt phẳng điều khiển mạng khỏi mặt phẳng dữ liệu. Mặt phẳng điều khiển đưa ra quyết định về cách các gói dữ liệu sẽ truyền qua mạng, mặt phẳng dữ liệu thực hiện việc di chuyển các gói từ nơi này sang nơi khác.

Một gói tin khi đến bộ chuyển mạch của mạng sẽ được các quy tắc được tích hợp trong phần sụn độc quyền của bộ chuyển mạch định hướng đến nơi chuyển tiếp gói. Các quy tắc xử lý gói này được gửi đến bộ chuyển mạch từ bộ điều khiển tập trung.

Các thiết bị chuyển mạch – còn được gọi là thiết bị mặt phẳng dữ liệu – truy vấn bộ điều khiển để nhận các chỉ thị và quy tắc định hướng khi cần thiết và cung cấp cho bộ điều khiển thông tin về lưu lượng mà nó xử lý. Switch sẽ gửi

mọi gói tin đến cùng một đích trên cùng một đường và xử lý tất cả các gói tin khác theo cùng một cách được chỉ định.

Chế độ hoạt động của SDN thường được gọi là thích ứng (adaptive) hoặc động (dynamic), trong đó một bộ chuyển mạch đưa ra yêu cầu định tuyến tới bộ điều khiển cho gói tin chưa được định tuyến cụ thể. Quá trình này khác với cơ chế định tuyến thích ứng trong các mạng truyền thống. Trong các mạng truyền thống, switch gửi các yêu cầu định tuyến thông qua các bộ định tuyến và thuật toán định tuyến của bộ định tuyến sẽ hoạt động dựa trên cấu trúc liên kết của mạng (topo mạng) mà không thông qua bộ điều khiển tập trung.

Khía cạnh ảo hóa của SDN phát huy tác dụng thông qua lớp phủ ảo (virtual overlay), là một mạng riêng biệt về mặt logic nằm trên mạng vật lý. Người dùng có thể triển khai các lớp phủ đầu cuối để trừu tượng hóa lưu lượng mạng và phân đoạn mạng cơ bản. Sự phân tách này đặc biệt hữu ích cho các nhà cung cấp và khai thác dịch vụ đám mây, vì họ có thể cung cấp một mạng ảo riêng tách biệt với các chính sách cụ thể cho từng bên thuê dịch vụ.

V. Lợi ích và thách thức của SDN

1. Lợi ích của SDN

- *Tính linh hoạt:*

Tính linh hoạt là lời giải đáp đầu tiên cho câu hỏi lợi ích của SDN là gì.

SDN cho phép các quản trị viên mạng dễ dàng cấu hình và điều chỉnh các thành phần của mạng thông qua phần mềm thay vì phải thay đổi phần cứng. Điều này giúp giảm thiểu thời gian và chi phí liên quan đến việc triển khai và bảo trì mạng.

Việc lập trình mạng bằng phần mềm cũng giúp cho phép người dùng phản ứng nhanh hơn với các yêu cầu kinh doanh thay đổi, mở rộng hoặc thu hẹp tài nguyên mạng một cách linh hoạt.

- *Quản lý tập trung:*

Một trong những lợi ích lớn nhất của SDN là khả năng quản lý tập trung. Thay vì phải cấu hình từng thiết bị mạng riêng lẻ, các quản trị viên có thể điều khiển toàn bộ mạng từ một điểm trung tâm thông qua các bộ điều khiển SDN.Qua đó, giúp đơn giản hóa quá trình quản lý, giảm thiểu lỗi cấu hình và tăng cường khả năng giám sát mạng.

- *Tối ưu hóa tài nguyên:*

SDN cho phép tối ưu hóa việc sử dụng băng thông và tài nguyên mạng bằng cách định tuyến lưu lượng một cách hiệu quả và tự động hóa các tác vụ quản lý mạng. Nhờ vào khả năng điều khiển linh hoạt, SDN giúp phân phối lưu lượng một cách hiệu quả hơn, giảm thiểu tắc nghẽn và cải thiện hiệu suất tổng thể của mạng.

- *Khả năng mở rộng:*

Khi cần thêm các thiết bị mới vào hệ thống, việc tích hợp vào mạng hiện tại trở nên đơn giản mà không cần thay đổi cấu trúc mạng phức tạp. Điều này đặc biệt quan trọng đối với các doanh nghiệp đang phát triển nhanh chóng và cần mở rộng mạng lưới một cách linh hoạt.

- *Bảo mật nâng cao:*

Với SDN, các chính sách bảo mật có thể được thực thi một cách đồng nhất và tự động trên toàn bộ mạng. Các quản trị viên có thể dễ dàng cập nhật và áp dụng các chính sách bảo mật mới mà không cần cấu hình lại từng thiết bị, giúp tăng cường bảo mật mạng và giảm thiểu rủi ro từ các mối đe dọa bảo mật.

- *Giảm chi phí vận hành:*

SDN giúp giảm chi phí vận hành bằng cách tự động hóa nhiều tác vụ quản lý mạng, giảm thiểu nhu cầu về nhân lực và giảm chi phí liên quan đến việc bảo trì phần cứng. Khả năng điều khiển tập trung và lập trình mạng bằng phần mềm

cũng giúp giảm thiểu lỗi và tăng cường hiệu quả vận hành, từ đó tiết kiệm chi phí cho doanh nghiệp.

- *Cải thiện hiệu suất, độ tin cậy:*

Nhờ vào khả năng tối ưu hóa lưu lượng và quản lý tài nguyên mạng một cách hiệu quả, SDN giúp cải thiện hiệu suất tổng thể của mạng. Khả năng phản ứng nhanh với các sự cố mạng và tự động điều chỉnh lưu lượng giúp duy trì độ tin cậy cao và giảm thiểu thời gian ngừng hoạt động.

- *Hỗ trợ triển khai các ứng dụng và dịch vụ mới:*

SDN còn cung cấp một nền tảng linh hoạt cho việc triển khai các ứng dụng và dịch vụ mới. Các quản trị viên có thể dễ dàng cấu hình và quản lý các dịch vụ mạng phức tạp mà không cần thay đổi phần cứng, giúp rút ngắn thời gian triển khai và đáp ứng nhanh chóng các yêu cầu kinh doanh mới.

- *Tích hợp dễ dàng với các công nghệ hiện đại*

SDN dễ dàng tích hợp với các công nghệ hiện đại như điện toán đám mây, Internet of Things (IoT) và ảo hóa mạng (NFV). Điều này giúp các doanh nghiệp tận dụng tối đa các lợi ích của công nghệ mới, đồng thời cải thiện hiệu quả và hiệu suất của hạ tầng mạng.

2. Thách thức của SDN

Vấn đề bảo mật không chỉ là lợi thế mà cũng là một thách thức đối với công nghệ SDN. SDN được quản lý và điều khiển thông qua một bộ điều khiển tập trung duy nhất, do đó, nó là một mục tiêu béo bở cho hacker. Một khi tấn công được thành công bộ điều khiển này kẻ tấn công có thể chiếm quyền điều khiển toàn mạng.

Một thách thức khác nghe có vẻ khá buồn cười với SDN đó là mặc dù nó có tên là "định nghĩa được" nhưng nó lại thực sự không có định nghĩa nào thống nhất và tiêu chuẩn chung. Các nhà cung cấp khác nhau cung cấp các cách tiếp

cận khác nhau cho SDN, từ các mô hình tập trung vào phần cứng và nền tảng ảo hóa cho đến các thiết kế mạng siêu hội tụ và các phương thức không sử dụng bộ điều khiển. Có rất nhiều các thiết kế và định hướng khác nhau. Do đó nó thường bị nhầm lẫn với khá nhiều khái niệm khác như: kết nối mạng hộp trắng (white box networking), phân hoạch mạng, tự động hóa mạng và mạng có thể lập trình. Mặc dù SDN có thể làm việc với các công nghệ và quy trình này, nhưng về bản chất nó vẫn là một công nghệ riêng biệt.

Công nghệ SDN nổi lên rất sớm và nhận được nhiều sự chú ý khoảng năm 2011, khi nó được giới thiệu cùng với giao thức OpenFlow. Tuy nhiên, kể từ đó, việc áp dụng nó vào thực tế diễn ra tương đối chậm, đặc biệt là trong số các doanh nghiệp có kiến trúc mạng nhỏ hơn và ít tài nguyên. Ngoài ra, nhiều doanh nghiệp đưa ra mức chi phí triển khai SDN rất cao là một yếu tố gây e ngại cho việc triển khai công nghệ này.

SDN đã được ứng dụng trong hệ thống mạng của các nhà cung cấp dịch vụ, nhà khai thác mạng, viễn thông, các nhà mạng, cùng với các công ty lớn, như Facebook và Google, tất cả đều có tài nguyên để giải quyết và đóng góp cho một công nghệ mới nổi. Tuy nhiên, sự phổ biến của SDN là chưa rộng do nhiều vấn đề đi kèm. Đặc biệt là chi phí triển khai ban đầu và kiến trúc hệ thống mạng hiện có của đa số doanh nghiệp vẫn chưa đủ lớn để cho thấy lợi ích thực sự của SDN.

CHƯƠNG II: TỔNG QUAN VỀ OPENFLOW

I. Giới thiệu về Openflow

Openflow là một trong những giao thức cốt lõi của SDN. Nó cho phép tạo ra các giao tiếp giữa bộ điều khiển và thiết bị chuyển mạch của mạng.

Ngày nay, các hệ thống mạng có cấu trúc ngày một phức tạp và trở lên ngày càng khó quản lý, tối ưu hơn. Nhu cầu về sự linh hoạt, tin cậy và an toàn của các hệ thống mạng cũng ngày càng cấp thiết. Tuy nhiên, các thiết bị mạng hiện nay vẫn đa phần được sản xuất đơn lẻ, không có tiêu chuẩn chung. Mỗi nhà cung cấp thường tự sản xuất và đưa ra các API cũng như cách thức cấu hình khác nhau. Điều này hạn chế khả năng phối hợp, điều khiển lưu lượng giữa các thiết bị của các nhà sản xuất khác nhau, khả năng tái lập trình và cấu hình mạng trở nên hạn chế. Trước tình hình đó khái niệm SDN ra đời. SDN giúp giải quyết vấn đề bằng cách tách biệt phần điều khiển thiết bị (control plan) và phần điều hướng dữ liệu (data plane).

Tuy nhiên, để tách biệt hai thành phần này mà vẫn đảm bảo sự đồng bộ cho toàn bộ các thiết bị là điều không hề dễ dàng. Đó là lí do giao thức Openflow được xây dựng. OpenFlow cung cấp một giao diện giao tiếp cho phép khả năng thực thi các điều khiển mức cao độc lập với phần cứng. OpenFlow là giao thức mạng cho phép điều khiển mạng xác định đường đi của gói tin của mạng qua các Switch. OpenFlow cho phép chuyển mạch từ các nhà cung cấp và các nhà phát triển khác nhau. Openflow đóng vai trò là lớp giao tiếp giữa phần điều khiển và phần chuyển tiếp và cung cấp các API cho phép khả năng lập trình cho lớp điều khiển. Open Networking Foundation (ONF) sử dụng giao thức OpenFlow. OpenFlow là tiêu chuẩn đầu tiên, cung cấp khả năng giao tiếp giữa các giao diện của lớp điều khiển và lớp chuyển tiếp trong kiến trúc SDN. OpenFlow cho phép truy cập trực tiếp và điều khiển lớp chuyển tiếp của các thiết bị mạng như switch và router, cả thiết bị vật lý lẫn thiết bị ảo. Trong OpenFlow, tất cả "phần thông minh" được đưa lên trên một server trung tâm, vì vậy nó thực hiện các hoạt động phức tạp một cách đơn giản hơn.

II. Lịch sử phát triển của OpenFlow:

Ngay từ khi ý tưởng về SDN ra đời, hàng loạt các công ty, tổ chức lớn trên thế giới như Google, Microsoft, Facebook, Cisco… đã đưa ra ý tưởng về một tiêu chuẩn chung thống nhất cho mạng SDN và là nền tảng để cho ra đời OpenFlow. OpenFlow có tiền thân là giao thức chuyển mạch OpenFlow Switching Protocol được đề xuất bởi tổ chức Open Networking Foundation (ONF).

Các nhà nghiên cứu từ đại học Stanford và đại học California bắt đầu phát triển SDN vào năm 2002, còn dự án OpenFlow được bắt đầu trong năm 2008. Juniper và các nhà cung cấp khác sản xuất các sản phẩm SDN độc quyền trên cơ sở của OpenFlow từ năm 2010. OpenFlow đã giúp cho SDN trở nên khả thi hơn. Tuy nhiên, do một số hạn chế về khả năng mở rộng mà các nhà phát triển khác nhau đã đẩy nó đi theo nhiều hướng khác nhau làm cho khả năng tương thích lẫn nhau của Openflow bị ảnh hưởng.

III. Các đặc trưng của OpenFlow:

- OpenFlow có thể được sử dụng bởi các ứng dụng phần mềm bên ngoài để điều khiển lớp chuyển tiếp của các thiết bị mạng, giống như tập lệnh của CPU điều kiển hệ thống máy tính.

- Giao thức OpenFlow được triển khai trên cả hai giao diện của kết nối giữa các thiết bị cơ sở hạ tầng mạng và phần mềm điều khiển SDN.

- OpenFlow sử dụng khái niệm “flow” (luồng) để nhận dạng lưu lượng mạng trên cơ sở định nghĩa trước các quy tắc phù hợp (được lập trình sẵn bởi phần mềm điều khiển SDN). Giao thức này còn cho phép định nghĩa các tham số, chẳng hạn như mô hình lưu lượng sử dụng, ứng dụng và tài nguyên cần thiết để xác định cách thức điều hướng lưu lượng truyền qua các thiết bị mạng. Do đó OpenFlow cho phép mạng được lập trình dựa trên cơ sở luồng lưu lượng. Một kiến trúc SDN trên cơ sở OpenFlow cung cấp điều khiển ở mức cực kỳ chi tiết, cho phép mạng phản hồi sự thay đổi theo thời gian thực của ứng dụng, người dùng và mức phiên. Mạng định tuyến trên cơ sở IP hiện tại không cung cấp mức

này của điều khiển, tất cả các luồng lưu lượng giữa hai điểm cuối phải theo cùng một đường thông qua mạng, mặc dù yêu cầu của chúng khác nhau.

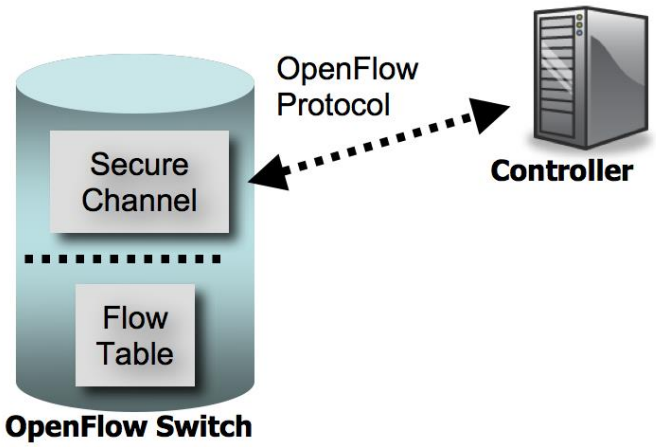
Giao thức OpenFlow là một chìa khóa để cho phép các mạng định nghĩa bằng phần mềm và cũng là giao thức tiêu chuẩn SDN duy nhất cho phép điều khiển lớp chuyển tiếp của các thiết bị mạng

IV. Kiến trúc và hoạt động của OpenFlow

1. Kiến trúc của OpenFlow

OpenFlow là tiêu chuẩn đầu tiên, cung cấp khả năng truyền thông giữa các giao diện của lớp điều khiển và lớp chuyển tiếp trong kiến trúc SDN. OpenFlow cho phép truy cập trực tiếp và điều khiển mặt phẳng chuyển tiếp của các thiết bị mạng như switch và router, cả thiết bị vật lý và thiết bị ảo, do đó giúp di chuyển phần điều khiển mạng ra khỏi các thiết bị chuyển mạch thực tế tới phần mềm điều khiển trung tâm. Các quyết định về các luồng traffic sẽ được quyết định tập trung tại OpenFlow Controller giúp đơn giản trong việc quản trị cấu hình trong toàn hệ thống.

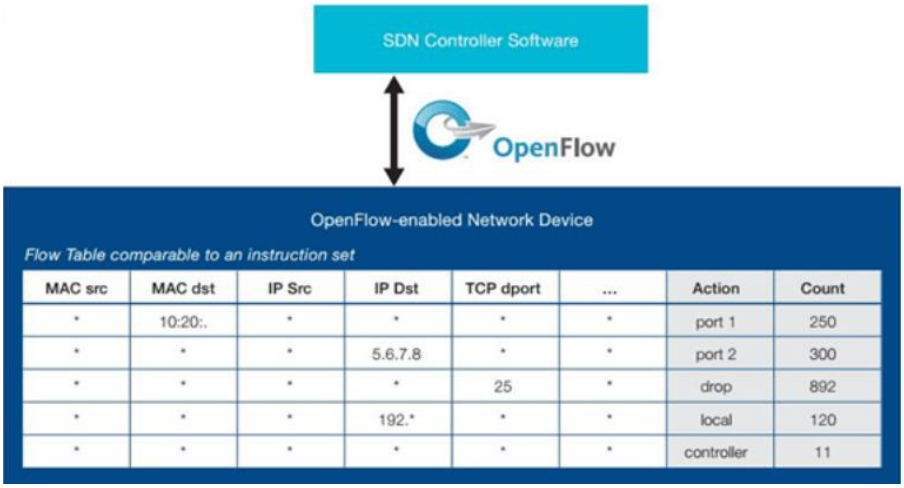
Một thiết bị OpenFlow bao gồm ít nhất 3 thành phần: Bảng luồng (Flow table), Kênh an toàn (Secure Channel), Giao thức OpenFlow (OpenFlow protocol)



Hình 4 Minh họa giao thức OpenFlow

a. Flow table

Một liên kết hành động với mỗi luồng, giúp thiết vị xử lý các luồng thế nào.



MAC src	MAC dst	IP Src	IP Dst	TCP dport	...	Action	Count
*	10:20:.	*	*	*	*	port 1	250
*	*	*	5.6.7.8	*	*	port 2	300
*	*	*	*	25	*	drop	892
*	*	*	192.*	*	*	local	120
*	*	*	*	*	*	controller	11

Hình 5 Bảng lưu lượng trong OpenFlow

b. Secure Channel

Kênh kết nối thiết bị tới bộ điều khiển (controller), cho phép các lệnh và các gói tin được gửi giữa bộ điều khiển và bộ thiết bị.

c. Openflow Protocol

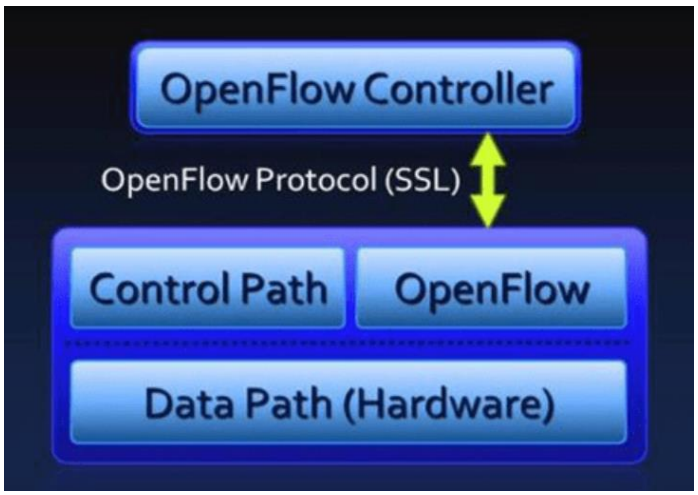
Giao thức cung cấp phương thức tiêu chuẩn và mở rộng cho một controller truyền thông với thiết bị.

2. Hoạt động của OpenFlow

Trong hầu hết các thiết bị chuyển mạch Ethernet hiện đại đều sử dụng các bảng lưu lượng (Flow tables). Những bảng này giúp cho việc chuyển các gói tin từ nơi gửi đến nơi nhận một cách hiệu quả. Mỗi nhà cung cấp sẽ có một bảng lưu lượng riêng, tuy nhiên ta vẫn có thể tách ra một tập hợp các chức năng, được xem là điểm chung cho tất cả các thiết bị chuyển mạch (Switch).

OpenFlow tách rời các chức năng của lớp truyền dữ liệu (Data Plane) và lớp điều khiển (Control Plane) ra khỏi nhau. Chức năng liên quan đến truyền dữ liệu vẫn được thực hiện trên thiết bị chuyển mạch (Switch) như cũ, còn các quyết

định về định tuyến cấp cao trong OpenFlow thì do bộ điều khiển (Controller) thực hiện. Controller và các Switch (thiết bị chuyển mạch) giao tiếp với nhau thông qua giao thức Giao thức chuyển mạch Openflow (OpenFlow Switching Protocol).



Hình 6 Cấu trúc Openflow Switch

Controller có thể ra lệnh cho các switch thực hiện các luật nhằm phân luồng dữ liệu mạng. Những luật này có thể là : truyền dữ liệu theo tuyến đường nhanh nhất, hoặc theo tuyến đường có ít hops nhất…

OpenFlow cung cấp giao diện API duy nhất, nhờ giao diện này người quản trị có thể lập trình công việc của mạng, và đồng thời có thể thiết lập các quy tác định tuyến gói tin, cân bằng tải, điều khiển truy nhập… Giao diện API này bao gồm 2 thành phần chính: Giao diện lập trình dành cho việc kiểm soát chuyển tiếp gói tin qua các bộ chuyển mạch mạng và bộ các giao diện toàn cầu (global interface), trên cơ sở những giao diện này có thể tạo ra các công cụ quản lý cấp cao.

V. Ưu điểm của OpenFlow

Openflow và SDN cho phép quản trị viên của mạng có khả năng giải quyết vấn đề quản lý băng thông và cấu hình cho mạng, cho phép xây dựng các mạng có băng thông cao, mềm dẻo dễ tương thích với các yêu cầu thay đổi về nghiệp vụ, tinh giản hóa các hoạt động quản lý phức tạp. Những lợi ích và ưu điểm mà các doanh nghiệp và nhà khai thác mạng có thể đạt được thông qua kiến trúc SDN trên cơ sở OpenFlow bao gồm:

- *Hiệu suất và chi phí:* Nhờ việc tách quá trình điều khiển và xử lý ra khỏi thiết bị chuyển mạch, OpenFlow cho phép những thiết bị này tận dụng toàn bộ tài nguyên của mình cho việc tăng tốc chuyển tiếp gói tin. Đồng thời nhờ ảo hóa sự điều khiển mạng, OpenFlow làm giảm chi phí trong việc xây dựng và hỗ trợ mạng.

- *Thực hiện và thử nghiệm các chức năng mới*: Công cụ phần mềm của OpenFlow cho phép người quản trị thêm chức năng mới vào kiến trúc mạng hiện có. Nhờ đó các chức năng mới sẽ làm việc trên nhiều nền tảng mà không cần tái thực hiện trong các firmware của thiết bị chuyển mạch của mỗi nhà cung cấp. Nhờ giao diện API mở, công nghệ OpenFlow cũng cho phép người quản trị viên hay lập trình viên tạo ra các phần mềm quản lý bất kỳ, từ đó thử nghiệm chức năng mới của thiết bị chuyển mạch. Trước đây, công việc này là rất khó khăn, vì các bộ định tuyến hay chuyển mạch được sản xuất bởi các nhà cung cấp khác nhau không có một giao diện API chung.

- *Bảo mật và quản lý dễ dàng*: Trên bộ điều khiển trung tâm (Controller) của OpenFlow, người quản trị có thể quan sát toàn bộ mạng dưới một cái nhìn duy nhất, nhờ đó tăng sự đơn giản trong điều khiển, hỗ trợ bảo mật và thực hiện các nhiệm vụ khác. Vì OpenFlow cho phép quản trị viên thấy rõ tất cả các luồng dữ liệu nên họ sẽ dễ dàng phát hiện sự xâm nhập trái phép hay làm rõ các vấn đề khác. OpenFlow đồng thời cũng cho phép người quản trị hệ thống thiết lập các ưu tiên đối với những dạng luồng dữ liệu khác nhau và phát triển các chính sách phù hợp cho mạng khi có sự cố tắc nghẽn hay các vấn đề khác với thiết bị. Ngoài ra, công nghệ OpenFlow hứa hẹn khả năng tạo ra cấu trúc mạng ảo, xây dựng

theo yêu cầu các mạng LAN và WAN ảo mà ko cần thay đổi cấu trúc phần cứng của mạng. Để thực hiện điều này cần phải xem xét khả năng tạo ra mặt phẳng điều khiển trung tâm ảo, hỗ trợ các chức năng quản lý mạng. Chức năng này có thể đặc biệt có ích cho việc điều khiển trung tâm xử lý dữ liệu. Ví dụ nhờ controller OpenFlow, người quản trị mạng có thể tạo ra mạng LAN ảo cho một khách hàng mới mà không cần phải thay đổi trong từng bộ chuyển mạch hay nhóm thiết bị chuyển mạch của một nhà cung cấp nhất định.

- *Điện toán đám mây:* Theo những người ủng hộ OpenFlow thì công nghệ này có khả năng hỗ trợ tốt các mức độ "thông minh" mong muốn của mạng cho điện toán đám mây.

CHƯƠNG III: CÁC VẤN ĐỀ AN TOÀN TRONG SDN

I. Các mối đe dọa và giải pháp

1. Giả mạo luồng traffic

Giả mạo luồng trafic, có thể được sử dụng để tấn công thiết bị chuyển mạch và bộ điều khiển. Mối đe dọa này có thể được kích hoạt bởi các thiết bị bị lỗi (không độc hại) hoặc bởi một người dùng độc hại. Một kẻ tấn công có thể sử dụng các phần tử mạng để khởi động một cuộc tấn công DoS chống lại OpenFlow switchs (ví dụ, nhắm mục tiêu để tràn bảng TCAMs) và các tài nguyên của controller.

Giải pháp: Việc sử dụng các hệ thống phát hiện xâm nhập với sự hỗ trợ cho việc phân tích thời gian truy cập hệ thống có thể giúp ta phát hiện các truy cập bất thường.

2. Khai thác lỗ hổng trong switch

Tấn công vào lỗ hổng trong switches, có thể dễ dàng tàn phá hệ thống mạng. Một switch có thể được sử dụng để drop hoặc làm chậm các gói tin trong mạng, hoặc đi chệch lưu lượng mạng (ví dụ, cho các mục đích trộm cắp dữ liệu), hoặc thậm chí tăng lưu lượng truy cập hoặc giả mạo yêu cầu quá tải bộ điều khiển hoặc thiết bị chuyển mạch lân cận.

Giải pháp: Việc sử dụng các cơ chế của phần mềm chứng thực, chẳng hạn như các giải pháp quản lý tin tưởng cho các thành phần phần mềm, là một yếu tố có thể giảm nhẹ. Cơ chế giám sát và phát hiện hành vi bất thường của các thiết bị mạng cũng có thể hữu ích để đánh bại loại mối đe dọa.

3. Khai thác lỗ hổng trong bộ điều khiển

Khai thác vào lỗ hổng trong bộ điều khiển, có thể là mối đe dọa nghiêm trọng nhất đối với SDN. Một bộ điều khiển bị lỗi hoặc độc hại có thể ảnh hưởng toàn bộ mạng, một ứng dụng độc hại có khả năng làm bất cứ điều gì nó thích trong mạng.

Giải pháp: Một số kỹ thuật có thể được sử dụng, chẳng hạn như sao chép (để phát hiện, loại bỏ hành vi bất thường), đa dạng sử dụng (các bộ điều khiển, giao thức, ngôn ngữ lập trình, hình ảnh phần mềm, v.v.), và phục hồi (định kỳ làm mới hệ thống về trạng thái an toàn và đáng tin cậy).

II. Triển khai mạng an toàn trong SDN

1. Triển khai - chuyển đổi từng bước thích hợp

Để an toàn với SDN, doanh nghiệp cần phải bắt đầu một cách thật chính xác. SDN là sự chuyển đổi từ một hệ thống mạng có thể cấu hình sang một hệ thống mạng có thể lập trình. Do đó, có một số bước mà doanh nghiệp buộc phải tuân theo.

Đầu tiên, doanh nghiệp phải tích hợp mọi thiết bị, gồm bộ cân bằng tải, tường lửa, hệ thống truyền thông, thiết bị không dây hoặc hệ thống lưu trữ… vào các bộ điều khiển SDN. Các nhà sản xuất SDN có các công cụ có thể di dời cấu hình hiện thời sang các SDN controller cho doanh nghiệp. Mọi thiết bị trên mạng đều phải hỗ trợ SDN. Đây là yếu tố rất quan trọng bởi vì bây giờ doanh nghiệp đang tạo ra một chức năng mạng SDN duy nhất, phủ khắp mọi thiết bị của mọi nhà sản xuất và nền tảng.

Bước thứ 2 là cẩn thận thử nghiệm và triển khai các bộ điều khiển SDN. Có thể tạo một môi trường thử nghiệm gồm các thiết bị mạng "nói chuyện" được với bộ điều khiển SDN - SDN controller để chắc chắn mọi thứ đều chạy tốt. Các thiết bị đều phải cùng chung một giao thức, hoặc OpenFlow hoặc OpenStack, và doanh nghiệp cần đảm bảo bước thứ 2 này là các thiết bị trong mạng và bộ điều khiển giao tiếp thành công với nhau.

Bước thứ 3 là bước quan trọng nhất, là bật SDN tương tác được với ứng dụng. Để gắn ứng dụng hoàn toàn tận dụng được mạng SDN, đầu tiên cần áp dụng các chính sách như ưu tiên lệnh thực thi cao hơn lệnh báo cáo để cho luồng dữ liệu quan trọng nhất chạy mượt mà trong hệ thống mạng. Các bộ điều khiển SDN sẽ phải nhận diện được các gói dữ liệu thông qua header mà không phải đọc toàn bộ dữ liệu để nhận diện loại dữ liệu.

2. Giải quyết những khó khăn, quan ngại

Một mạng SDN được triển khai đúng cách, sử dụng các biện pháp bảo mật thích hợp có thể loại trừ được các quan ngại bảo mật trước kia về SDN. Ví dụ, vài chuyên gia bảo mật e sợ SDN bỏ đi lớp phần cứng như tường lửa. Nhưng SDN thực sự giám sát các gói dữ liệu ở nhiều tầng khác nhau trong mạng, từ thiết bị cân bằng tải đến tường lửa, ứng dụng, bộ điều khiển LAN không dây cho đến lưu trữ và hệ thống điện thoại.

Mạng nên đảm bảo độ an toàn cho bộ điều khiển này bằng cách cẩn thận cài các bản vá và cập nhật, tăng điều khiển máy chủ lên cấp AAA. Đó là thành phần tối quan trọng trong cơ sở hạ tầng, nó cần nhà quản trị quan tâm ở mức cao hơn thông thường.

CHƯƠNG IV: CÁC BỘ ĐIỀU KHIỂN - SDN CONTROLLER PHỔ BIẾN

I. Các bộ điều khiển trong SDN

1. OpenDayLight

a. Giới thiệu

Bộ điều khiển SDN nguồn mở OpenDaylight là một trong những nền tảng *SDN controller* thành công xuất hiện trong những năm 2010. OpenDaylight đã sử dụng nhiều nguyên tắc nguồn mở giống nhau được sử dụng với Linux. Ý tưởng của OpenDaylight là nếu có đủ các nhà cung cấp làm việc cùng nhau trên một bộ điều khiển nguồn mở chung thì tất cả sẽ có lợi. Tất cả các nhà cung cấp sau đó có thể sử dụng bộ điều khiển nguồn mở làm cơ sở cho các sản phẩm của riêng họ, với mỗi nhà cung cấp tập trung vào phần khác biệt của sản phẩm, hơn là các tính năng cơ bản. Kết quả là vào giữa những năm 2010, bộ điều khiển SDN OpenDaylight (www.opendaylight.org) đã ra đời. OpenDaylight (ODL) bắt đầu như một dự án riêng biệt nhưng bây giờ tồn tại dưới dạng một dự án được quản lý bởi Linux Foundation.

b. Tính năng

- OpenDayLight là phần mềm mã nguồn mở dành cho Software Defined Networking (SDN) sử dụng giao thức mở cung cấp khả năng kiểm soát tập trung, có khả năng lập trình được và theo dõi các thiết bị mạng. Giống như nhiều SDN Controllers khác, OpenDaylight hỗ trợ OpenFlow, cũng như cung cấp các giải pháp mạng khác sẵn sàng để cài đặt khi có yêu cầu.

- OpenDaylight cung cấp giao diện cho phép kết nối các thiết bị mạng nhanh chóng và thông minh để tối ưu hiệu năng mạng

- OpenDaylight Controller cung cấp Northbound APIs, được sử dụng bởi các ứng dụng. Các ứng dụng này sử dụng controller để thu thập thông tin về mạng, chạy các thuật toán để kiểm soát, phân tích, sau đó sử dụng OpenDaylight Controller tạo các rules mới cho mạng.

- OpenDaylight Controller viết bằng ngôn ngữ Java, có nghĩa là có thể sử dụng OpenDaylight Controller trên bất kì môi trường nào hỗ trợ Java. Tuy nhiên để đạt hiệu năng tốt nhất, OpenDaylight nên chạy trên môi trường Linux hỗ trợ JVM tối thiểu 1.7.

2. Open Network Operating System (ONOS)

a. Giới thiệu

ONOS là một controller SDN mã nguồn mở được thiết kế cho các nhà mạng và các nhà cung cấp dịch vụ, tập trung vào hiệu suất cao, khả năng mở rộng, và độ tin cậy. Nó được phát triển bởi Open Networking Lab (ON.Lab) với mục tiêu hỗ trợ các ứng dụng SDN trên quy mô lớn.

b. Tính năng

- *Khả năng chịu lỗi cao*: ONOS được thiết kế để hoạt động ổn định ngay cả trong môi trường mạng lớn, nơi cần độ tin cậy và khả năng khôi phục nhanh khi xảy ra sự cố.

- *Mở rộng quy mô:* ONOS có thể mở rộng quy mô để hỗ trợ hàng triệu thiết bị mạng, phù hợp với các nhà mạng lớn và môi trường viễn thông.

- *Hỗ trợ multi-tenancy:* ONOS hỗ trợ nhiều người dùng khác nhau (multi-tenancy), cho phép các dịch vụ mạng hoạt động trên nhiều môi trường khác nhau mà không xung đột.

- *Northbound và Southbound APIs:* ONOS cung cấp cả hai loại API để tương tác với các ứng dụng và thiết bị mạng.

- *Hiệu suất cao:* Được tối ưu hóa để quản lý hiệu quả mạng lớn và phức tạp.

3. RYU

a. Giới thiệu

RYU là một controller SDN mã nguồn mở được viết bằng Python, dễ sử dụng và linh hoạt. RYU cung cấp một nền tảng lập trình để phát triển các ứng dụng SDN, đặc biệt tập trung vào việc hỗ trợ đầy đủ OpenFlow.

b. Tính năng

- *Hỗ trợ toàn diện OpenFlow:* RYU hỗ trợ các phiên bản khác nhau của OpenFlow, giúp nó trở thành công cụ lý tưởng để phát triển và thử nghiệm các ứng dụng SDN.

- *API dễ sử dụng:* Do được viết bằng Python, RYU có một API thân thiện và dễ tiếp cận, cho phép các nhà phát triển dễ dàng tạo ra các ứng dụng mạng tùy chỉnh.

- Kiến trúc mô-đun: Giống như nhiều SDN controllers khác, RYU cho phép người dùng thêm các mô-đun tùy chỉnh để mở rộng chức năng.

- *Hỗ trợ phát triển nhanh*: Do đơn giản và dễ tiếp cận, RYU phù hợp cho các dự án phát triển nhanh, thử nghiệm ý tưởng, và nghiên cứu.

- *Cộng đồng phát triển mạnh mẽ:* RYU có một cộng đồng lớn và hoạt động tích cực, với nhiều tài liệu và công cụ hỗ trợ.

4. POX

a. Giới thiệu

POX là một controller SDN đơn giản, chủ yếu được sử dụng cho mục đích giáo dục và nghiên cứu. Nó được viết bằng Python và cung cấp một nền tảng để phát triển các ứng dụng SDN cơ bản.

b. Tính năng

- *Hỗ trợ OpenFlow:* POX hỗ trợ các phiên bản cũ hơn của OpenFlow, làm cho nó phù hợp với các dự án nhỏ và các ứng dụng thử nghiệm.

- *Dễ dàng cấu hình và sử dụng:* POX có thiết kế đơn giản, dễ dàng thiết lập và sử dụng cho các ứng dụng và mô phỏng mạng nhỏ.

- *Thân thiện với người học:* Nhờ sự đơn giản và tài liệu phong phú, POX là lựa chọn phổ biến cho các nhà nghiên cứu và sinh viên khi học về SDN và OpenFlow.

- *Tùy biến linh hoạt:* Người dùng có thể phát triển và thử nghiệm các tính năng mạng mới bằng cách thêm các mô-đun vào POX.

- *Cộng đồng hỗ trợ:* Mặc dù không còn được phát triển mạnh mẽ, POX vẫn có một cộng đồng hỗ trợ với nhiều tài liệu hữu ích cho các nhà phát triển.

II. Mininet

1. Định nghĩa

Mininet là một công cụ giả lập mạng mạnh mẽ, cho phép tạo ra một mạng ảo bao gồm các thành phần như host (máy đầu cuối), switch, router và các liên kết trên nền tảng Linux kernel. Mininet sử dụng công nghệ ảo hóa đơn giản để tạo nên một hệ thống mạng hoàn chỉnh, chạy trên cùng một hệ điều hành, kernel và user code. Điều này giúp người dùng kiểm tra các ứng dụng và cấu hình mạng trên các mạng thực tế mà không cần phải có phần cứng vật lý.

2. Tính năng

- *Mạng ảo hóa hoàn chỉnh:* Mininet có thể tạo ra các host, switch, router và controller như trong một mạng thật nhưng dưới dạng phần mềm. Các host ảo có thể thực thi các ứng dụng thực như web servers, TCP monitoring, và các công cụ phân tích gói tin như Wireshark.

- *Dễ dàng cài đặt và sử dụng:* Mininet không yêu cầu cấu hình đặc biệt về phần cứng. Nó có thể được cài đặt và chạy trên laptop, server, VM, hoặc trong môi trường đám mây (cloud) sử dụng hệ điều hành Linux.

- *Tùy chỉnh topo mạng:* Mininet cho phép người dùng tạo, tùy chỉnh và thay đổi các topo mạng một cách nhanh chóng và linh hoạt. Người dùng có thể kiểm tra các thiết kế mạng khác nhau và cấu hình chuyển tiếp gói tin.

- *Hiệu năng tốt và độ chính xác cao:* Mininet có khả năng mô phỏng lưu lượng mạng với các thông số về tốc độ liên kết và độ trễ, giúp người dùng thực hiện các thử nghiệm với độ chính xác cao.

- *Khả năng tương tác với phần mềm thực:* Các host trên Mininet có thể chạy phần mềm thực sự như đã cài trên hệ thống Linux, giúp kiểm thử các ứng dụng mạng trong môi trường ảo mà không phải cần mạng vật lý.

- *Hỗ trợ các nghiên cứu và phát triển SDN:* Mininet đặc biệt hữu ích trong nghiên cứu và phát triển mạng SDN (Software Defined Networking), cung cấp môi trường lý tưởng để thử nghiệm các giao thức điều khiển và kiểm tra hiệu suất mạng.

CHƯƠNG V: HƯỚNG PHÁT TRIỂN

I. Đề xuất phương án:

Phướng án 1: Sử dụng 2 máy ảo

- **Máy ảo 1:** Chạy Mininet để mô phỏng toàn bộ mạng SDN, bao gồm các host và switch.

  - Máy ảo này sẽ thực hiện việc giả lập topology mạng, bao gồm các switch và host trong môi trường Mininet.
  - Mininet sẽ thực hiện các mô phỏng về quá trình truyền tải dữ liệu qua các switch sử dụng giao thức OpenFlow.

- **Máy ảo 2:** Chạy controller (POX hoặc Ryu)

  - Đây là controller chính để quản lý và điều khiển luồng dữ liệu trong mạng thông qua giao thức OpenFlow.
  - Controller sẽ đưa ra các quyết định liên quan đến cách thức truyền tải dữ liệu, bao gồm các luồng multicast.

Phương án 2: Sử dụng 1 máy ảo

Nếu bạn có một máy tính với cấu hình mạnh, có thể sử dụng một máy ảo để chạy cả Mininet và controller trên cùng một hệ thống.

- Trong trường hợp này, máy ảo sẽ đồng thời chạy Mininet và controller. Mininet sẽ tạo ra mạng giả lập, trong khi controller sẽ quản lý các luồng dữ liệu trong mạng.

II. Lựa chọn phương án:

Sau khi xem xét các phương án khả thi để triển khai môi trường mạng định nghĩa phần mềm (SDN), chúng tôi đã quyết định chọn **Phương án 1: Sử dụng hai máy ảo**. Quyết định này dựa trên các lý do sau:

- **Tối ưu hóa hiệu suất:** Việc sử dụng hai máy ảo tách biệt cho Mininet và controller cho phép tối ưu hóa hiệu suất hoạt động của từng thành phần. Máy ảo chạy Mininet sẽ chuyên biệt cho việc giả lập mạng, trong khi máy ảo chạy

controller sẽ đảm nhận nhiệm vụ quản lý luồng dữ liệu, từ đó giúp giảm thiểu tình trạng quá tải.

- Khả năng mở rộng: Phương án này tạo điều kiện thuận lợi cho việc mở rộng trong tương lai. Nếu cần bổ sung thêm các chức năng hoặc thành phần khác, việc phân chia thành hai máy ảo sẽ dễ dàng hơn trong việc cấu hình và quản lý.

- Dễ dàng kiểm soát và bảo trì: Việc tách biệt các thành phần của môi trường SDN giúp đơn giản hóa quá trình kiểm soát và bảo trì. Các lỗi phát sinh trong một máy ảo sẽ không ảnh hưởng đến hoạt động của máy ảo còn lại, từ đó giúp nâng cao độ tin cậy của hệ thống.

- Mô phỏng chính xác hơn: Sử dụng hai máy ảo cho phép thực hiện các mô phỏng chính xác về quá trình truyền tải dữ liệu qua các switch OpenFlow, giúp đánh giá hiệu suất và tính khả thi của các giải pháp SDN một cách hiệu quả.

CHƯƠNG VI: DEMO

I. Mục tiêu:

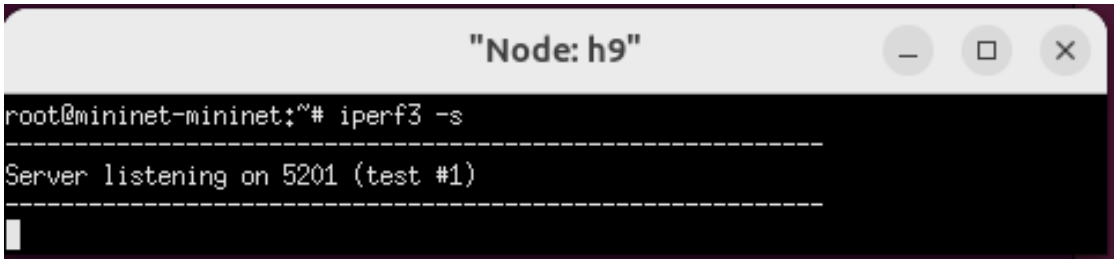
Mô phỏng thành công topo mạng SDN và đo băng thông trên Mininet trước và sau khi tấn công DDOS

II. Các yêu cầu cài đặt và công cụ:

- 2 máy Ubuntu: 1 máy cài đặt Mininet và 1 máy cài đặt Controller (ODL)
- Iperf: Dùng để đo băng thông trên mang SDN trước và sau khi tấn công
- Hping3: Công cụ dùng để tấn công DDOS

III. Quá trình thực hiện đo iperf3 khi bị tấn công bằng hping3

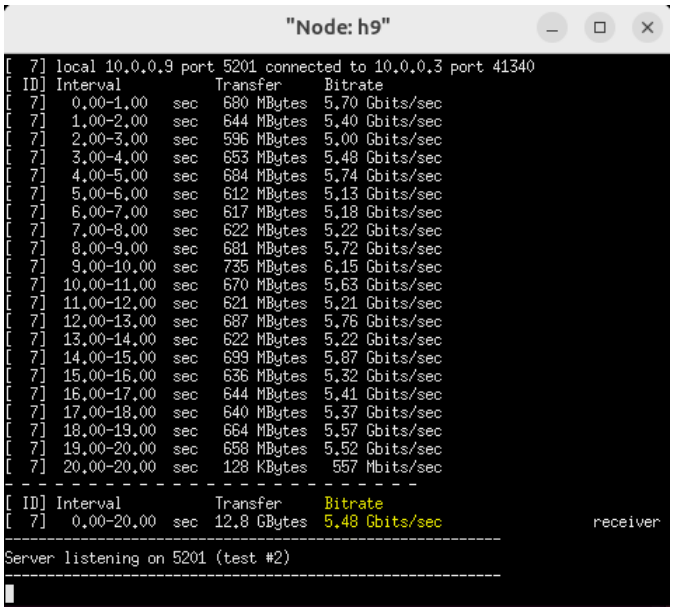
Bước 1: Chạy iperf3 Server trên h9



Hình 9 Chạy iperf3 Server trên Host 9

Bước 2: chạy iperf3 client trên h3 để đo băng thông trước khi tấn công hping3:

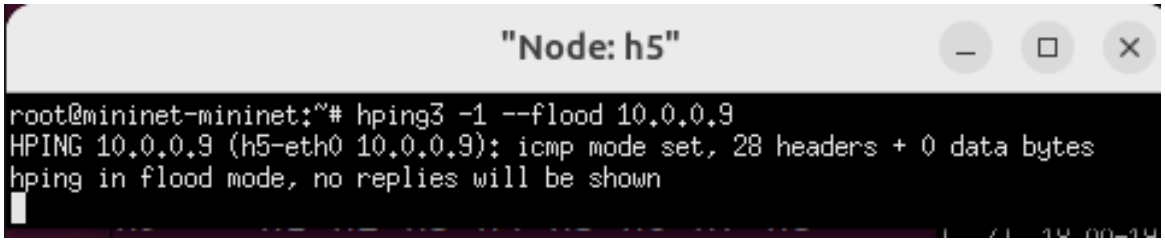
--option -t để set thoi gian thực hiện đo (theo giây), kh set thì mặc định là 10s-



Xem kết quả từ Iperf Server (h9) : Băng thông lúc này là 5,49 Gbits/sec

Hình 10 Host 9 trước khi bị tấn công

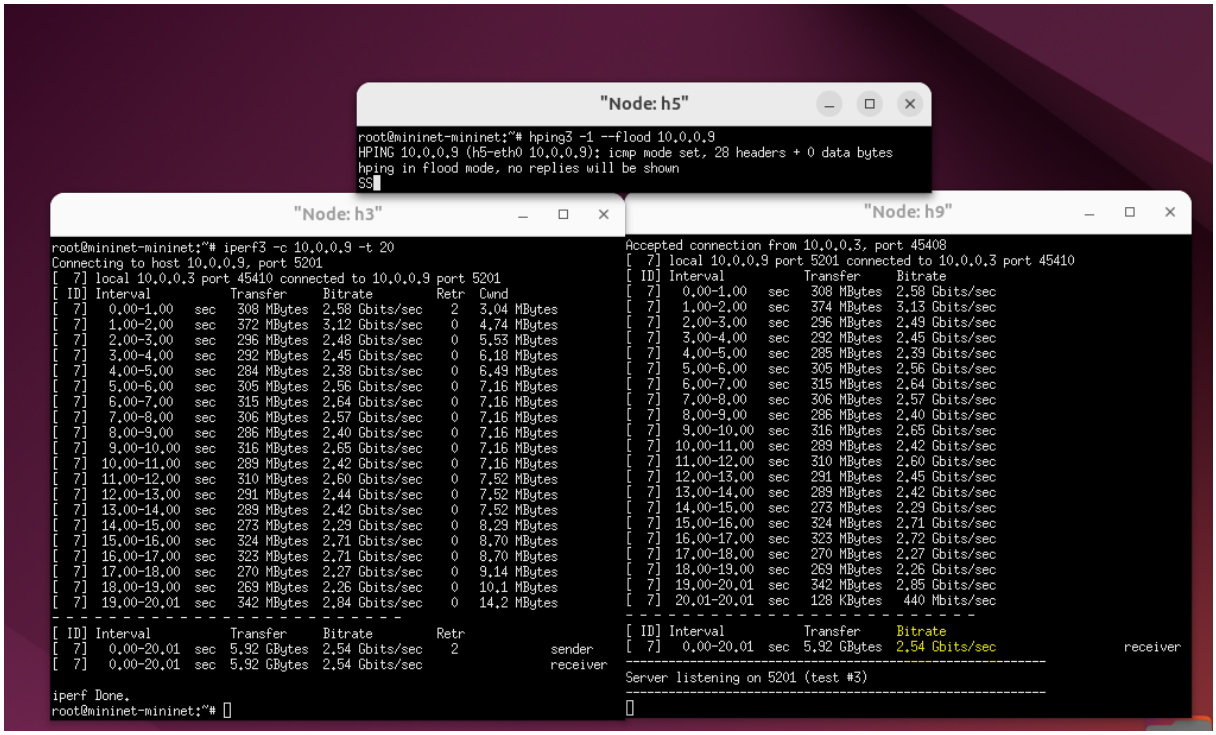
Bước 3: Thực hiện tấn công truyền gói tin ICMP (option --flood) bằng hping3 trên 1 host khác (h5), để tạo ra sự quá tải lưu lượng mạng.



Hình 11 Thực hiện tấn công lên Host 9

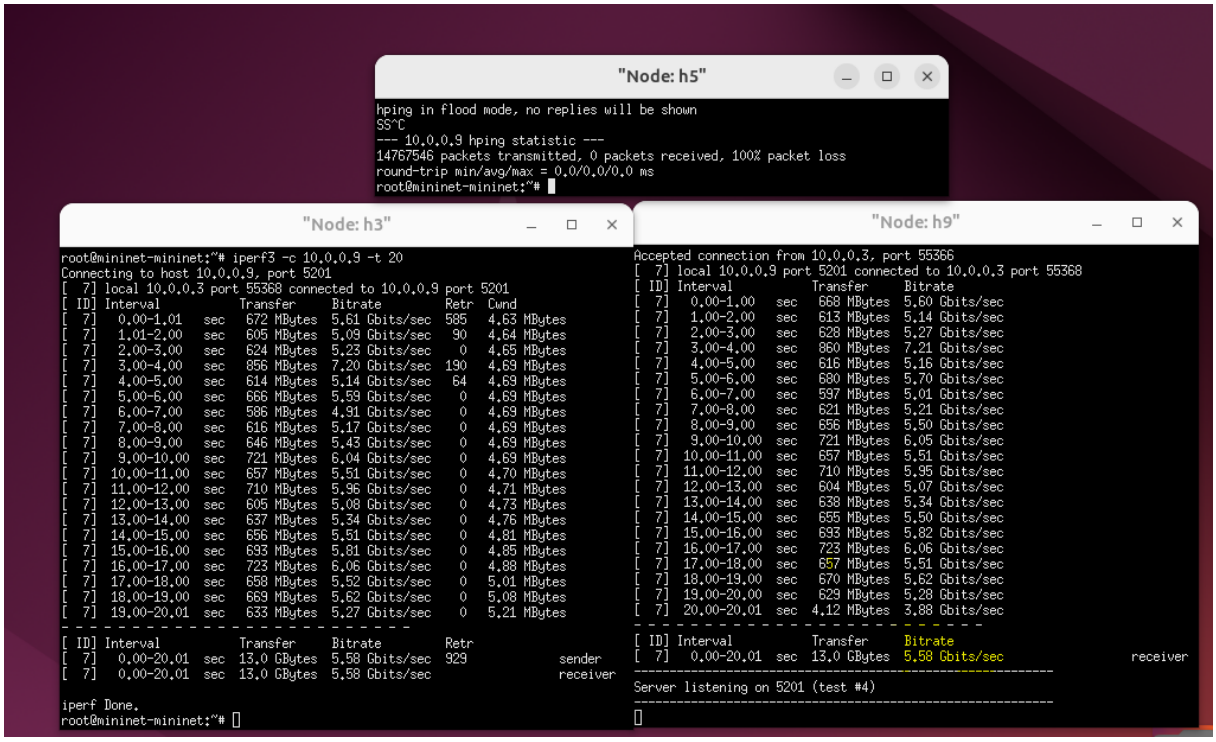
Bước 4:

- Đo băng thông giữa h1 và h9 trong khi h5 đang tấn công hping3.
- Kết quả lúc này băng thông giữa h1 và h9 đã giảm còn 2,54 Gbits/sec



Hình 12 Kết quả sau khi bị tấn công

Bước 5: Ngừng tấn công, bằng thông giữa h1 và h9 tăng trở lại 5.58 Gbits/sec



Hình 13 Kết quả sau khi ngừng tấn công

CHƯƠNG VI: HƯỚNG PHÁT TRIỂN

Theo yêu cầu của thầy thì phần này chúng em xin phép được trình bày hướng phát triển theo hướng thực tế. Vấn đề được đặt ra ở đây là có thể làm những gì để mô hình này được áp dụng trên môi trường thực. Để triển khai mô hình SDN vào hệ thống mạng thực tế dựa trên tiêu chuẩn OpenFlow, chúng ta có thể áp dụng các bước sau:

1. Cấu Hình và Kết Nối Switch với Controller qua OpenFlow

Thiết lập các switch hỗ trợ OpenFlow (vật lý hoặc ảo) kết nối với controller SDN (như Ryu hoặc OpenDaylight) qua **OpenFlow Channel**. Giao thức OpenFlow cho phép controller từ xa quản lý các switch, gửi các lệnh điều khiển tới các flow tables và group tables của switch, từ đó kiểm soát quá trình truyền dữ liệu.

Trên switch vật lý hoặc thiết bị ảo hóa như Mininet, định nghĩa các loại cổng như cổng vật lý (physical port) cho các kết nối mạng chính, và **cổng logic** cho các kết nối đặc biệt như các giao thức định tuyến hoặc loopback. Điều này đảm bảo tính linh hoạt khi kết nối các switch với controller, đồng thời tối ưu hóa luồng dữ liệu.

2. Thiết Kế Hệ Thống Pipeline và Flow Table cho Xử Lý Dữ Liệu

Xây dựng các **flow tables** và **group tables** trên switch để định tuyến gói tin dựa trên các trường hợp thực tế trong mạng. Sử dụng các **flow entry** trong flowtable để xác định các lệnh điều khiển (instruction) và hành động (action) xử lý trên mỗi gói tin dựa vào các trường như địa chỉ IP, VLAN, v.v.

Để đáp ứng các nhu cầu cụ thể của hệ thống mạng, thiết lập pipeline trong OpenFlow để luồng dữ liệu đi qua các bảng một cách tuần tự. Pipeline sẽ quyết định hành động cho các gói tin ở từng bước (chẳng hạn như chuyển tiếp, bỏ qua, hoặc điều chỉnh metadata của gói tin).

3. Thiết Lập và Quản Lý Nhóm Hành Động (Group Table)

Thiết kế **group table** cho các tình huống cần hành động phức tạp, chẳng hạn như phát tán gói tin đến nhiều điểm đích hoặc thực hiện phân phối tải. Các group này được chia thành các loại như:

- **All group**: Sử dụng cho truyền phát đa hướng hoặc phát tán dữ liệu đến tất cả các cổng đầu ra được cấu hình.
- **Select group**: Dùng cho việc lựa chọn ngẫu nhiên cổng đầu ra nhằm phân phối tải.
- **Fast Failover group**: Hỗ trợ phục hồi nhanh khi có sự cố, chọn cổng đầu ra thay thế khi cổng chính bị lỗi.

Trong môi trường thực tế, việc sử dụng các group này giúp tối ưu hóa lưu lượng mạng, đảm bảo độ tin cậy, và giảm thiểu thời gian phục hồi sau sự cố.

4. Thiết Lập Meter Table và Quản Lý Chất Lượng Dịch Vụ (QoS)

Sử dụng **meter tables** để kiểm soát tốc độ truyền dữ liệu cho từng luồng (flow) và thực hiện các chính sách quản lý băng thông nhằm đảm bảo chất lượng dịch vụ. Meter table có thể giới hạn tốc độ truyền hoặc gán các mức ưu tiên khác nhau cho từng loại lưu lượng (thông qua các loại meter band như drop hoặc DSCP remark).

Điều này sẽ giúp mạng xử lý lưu lượng với hiệu suất cao hơn, hạn chế tình trạng quá tải trên các đường truyền chính và đảm bảo chất lượng cho các dịch vụ quan trọng.

5. Giám Sát và Điều Chỉnh Động Thông Qua Controller

Tận dụng các asynchronous messages giữa controller và switch để nhận thông báo về các sự kiện như thay đổi trạng thái cổng, phát hiện lỗi, hoặc gói tin không khớp. Controller có thể phản hồi với các điều chỉnh động cho các flow entries hoặc thực hiện tái định tuyến khi cần thiết.

Điều này tạo điều kiện để controller không chỉ quản lý lưu lượng một cách linh hoạt mà còn nhanh chóng phản ứng với các sự cố trong mạng, duy trì tính ổn định và hiệu quả cho hệ thống.

6. Mở Rộng Mạng SDN với Các Switch Hybrid và OpenFlow-hybrid

Với các switch hỗ trợ OpenFlow-hybrid, chúng ta có thể thiết lập các chính sách phân loại lưu lượng, cho phép một phần dữ liệu được xử lý qua pipeline OpenFlow, trong khi phần còn lại được xử lý qua pipeline truyền thống của switch. Điều này đặc biệt hữu ích trong các mạng chuyển đổi từ mô hình truyền thống sang SDN.

Ứng dụng switch OpenFlow-hybrid trong thực tế cho phép các tổ chức vừa tận dụng các tính năng hiện có, vừa chuẩn bị cho việc chuyển đổi dần sang SDN toàn diện.

7. Kết Luận

Hướng phát triển này sẽ giúp chuyển đổi các khái niệm lý thuyết về OpenFlow thành các bước áp dụng thực tế để xây dựng một hệ thống mạng SDN hiệu quả, đáp ứng yêu cầu linh hoạt, dễ quản lý và tối ưu chi phí.

KẾT LUẬN

Trong bối cảnh công nghệ mạng ngày càng phát triển, mạng SDN (Software Defined Network) đã nổi lên như một giải pháp ưu việt, mang lại nhiều lợi ích trong việc quản lý và tối ưu hóa hạ tầng mạng. Qua quá trình tìm hiểu các mô hình mạng SDN, chúng em đã nhận thấy rằng SDN không chỉ giúp tách biệt các chức năng điều khiển và chuyển tiếp dữ liệu, mà còn thúc đẩy khả năng ảo hóa và tự động hóa trong quản lý mạng.

Việc triển khai môi trường giả lập SDN với Mininet đã giúp chúng em có cái nhìn sâu sắc về cách thức hoạt động của SDN, từ việc tạo ra các thiết bị ảo cho đến việc cấu hình và quản lý lưu lượng mạng. Mininet cung cấp một nền tảng linh hoạt, cho phép mô phỏng các tình huống thực tế và kiểm tra hiệu suất của mạng SDN trong môi trường an toàn và dễ dàng.

Những kết quả thu được từ quá trình này không chỉ khẳng định tiềm năng của SDN trong việc cải thiện hiệu quả và tính linh hoạt của mạng, mà còn mở ra hướng đi mới cho các nghiên cứu và ứng dụng trong tương lai. SDN, với khả năng lập trình và khả năng thích ứng linh hoạt, sẽ ngày càng trở thành một phần không thể thiếu trong các hạ tầng mạng hiện đại, phục vụ cho các nhu cầu ngày càng cao của người dùng và doanh nghiệp.

Cuối cùng, với những hiểu biết và kinh nghiệm đã tích lũy được từ việc tìm hiểu và triển khai SDN, chúng em có thể tự tin áp dụng kiến thức này vào các dự án thực tiễn trong lĩnh vực mạng, góp phần vào sự phát triển bền vững của công nghệ thông tin trong thời đại số.

Xin cảm ơn thầy *Lương Minh Huấn* đã hướng dẫn và hỗ trợ chúng em trong suốt quá trình thực hiện đề tài này.

TÀI LIỆU THAM KHẢO

[1] Nguyễn Minh Nhật – TP. GP Mạng HCM, HIS. "Chuyên đề Software Defined Network – Phần 2: SDN-Từ cảm hứng đến bản chất". 23/02/2021,

https://hpt.vn/tin-tuc/chuyen-de-software-defined-network-%E2%80%93-phan-2-sdn-tu-cam-hung-den-ban-chat/986

[2] Ngo The Hai Anh. "Tổng quan về SDN và OpenvSwitch". 2017,

https://viblo.asia/p/tong-quan-ve-sdn-va-openvswitch-m68Z0N865kG

[3] Tek4.vn. "SDN là gì? Tại sao SDN lại là xu thế cho công nghệ mạng tương lai?". 2022,

https://tek4.vn/sdn-la-gi-tai-sao-sdn-lai-la-xu-cho-cong-nghe-mang-tuong-lai

[4] Viettel IDC. "SDN là gì? Khám phá cách thức hoạt động và lợi ích của mạng lưới định nghĩa phần mềm". 23/05/2024,

https://viettelidc.com.vn/tin-tuc/sdn-la-gi-kham-pha-cach-thuc-hoat-dong-va-loi-ich

[5] Tek4.vn. "Giao thức OpenFlow", 17/04/2022.

https://tek4.vn/giao-thuc-openflow

[6] ThaiPH. "Software-defined Networking và OpenFlow", 2016.

https://github.com/hocchudong/thuctap032016/blob/master/ThaiPH/SDN/ThaiPH_SDN_OpenFlow.md

[7] LeBa Ba . "Tìm hiểu các vấn đề an toàn trong SDN"

https://www.academia.edu/13497161/T%C3%ACm_hi%E1%BB%83u_c%C3%A1c_v%E1%BA%A5n_%C4%91%E1%BB%81_an_to%C3%A0n_trong_SDN

[8] ThaiPH. “Lab SDN - OpenDaylight SDN controller với Mininet Emulator”, 2016.

https://github.com/hocchudong/thuctap032016/blob/master/ThaiPH/SDN/ThaiPH_opendaylight_mininet_lab.md#mininet

[9] Open Networking Foundation. *OpenFlow Switch Specification, Version 1.3.1 (Wire Protocol 0x04)*. 6 tháng 9, 2012.

https://www.opennetworking.org/standards/intro-to-openflow

[10] Nguyễn Minh Nhật. *Chuyên đề Software Defined Network – Phần 3: SDN - Từ bản chất đến thực tiễn*. Ngày 18/06/2021.

https://hpt.vn/tin-tuc/chuyen-de-software-defined-network-%E2%80%93-phan-3-sdn-tu-ban-chat-den-thuc-tien/1024.

TÀI LIỆU HƯỚNG DẪN CÀI ĐẶT

Tiến hành cài đặt môi trường demo SDN

Hướng dẫn cài đặt SDN với OpenDayLight và Mininet với 2 máy VM workstation.

link tai mininet: https://github.com/mininet/mininet/releases

(dowload and import bản image cài sẵn mininet)

Link tải Ubuntu: https://ubuntu.com/download/desktop

(dowload bản desktop (GUI) hoặc bản Server (dòng lệnh))

Cài đặt MobaXTerm cho máy thật (mục đích dùng máy thật Remote các máy ảo trên VM, để thuận tiện cho copy lệnh.)

Note: lệnh ip addr check địa chỉ ip để connect bằng mobaXTerm.

Cách làm:

- Set cả 2 máy card mạng NAT để ra Internet.
- 1 máy Ubuntu cài đặt OpenDayLight làm controller.-
- 1 máy (Image) Mininet.
- Cài đặt ssh:

```
sudo apt-get update
sudo apt-get install openssh-server
sudo apt-get install openssh-client
sudo systemctl start ssh.service
sudo systemctl restart ssh.service
```

- **Cài đặt Java: (để cài đặt OpenDayLight)**

+ link hướng dẫn cài đặt java: https://john.soban.ski/how-to-install-opendaylight-on-centos-or-ubuntu.html

+ Cài đặt JAVA:

```
sudo apt-get -y install openjdk-8-jre
sudo update-alternatives --config java
```

Vào file bashrs bằng lệnh:

```
nano  ~/.bashrc
```

Thêm dòng dưới vào cuối file `bashrc`:

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64/jre
```

Lưu lại tiến trình sau khi chỉnh sửa:

```
source  ~/.bashrc
```

Kiểm tra xem đã cài đặt môi trường JAVA thành công chưa.)

```
echo $JAVA_HOME
```

- **Cài đặt OpenDayLight (ODL)**

+ Lệnh dowload ODL:

```
sudo wget
https://nexus.opendaylight.org/content/groups/public/org/opendaylight/integration
/karaf/0.8.4/karaf-0.8.4.tar.gz
```

+ Lệnh giải nén:

```
tar -xzf karaf-0.8.4.tar.gz
```

+ Lệnh chạy (khởi động) ODL, Vào thư mục bin và chạy bằng lệnh:

```
./karaf
```

+ Lệnh thao tác trong karaf (cài cái feature cần thiết)

Các lệnh cài feature để xem giao diện topo mạng:

```
feature:list --installed
feature:install odl-l2switch-switch-ui
```

1 số feature khác (không cần dùng trong đồ án):

```
feature:install odl-dluxapps-applications odl-dluxapps-nodes odl-dluxapps-topology odl-dluxapps-yangui odl-dluxapps-yangman odl-dluxapps-yangvisualizer odl-dluxapps-yangutils odl-dlux-core
feature:install odl-restconf odl-l2switch-switch
feature:install odl-l2switch-all
```

Note:

Lệnh kiểm tra các feature đã cài đặt:

```
feature:list | grep Installed
```

Lệnh gỡ cài đặt feature:

```
feature:uninstall <tên-feature>
```

- Cài đặt các công cụ cần thiết khác:

+ Snort (công cụ phát hiện xâm nhập):

```
sudo apt-get install snort
```

+ Hping3 (công cụ tạo lưu lượng tấn công)

```
sudo apt-get install hping3
```

+Wireshark

```
sudo apt-get install tcpdump wireshark
```

- Thiết lập cấu hình các switch và host (thực hiện bên máy mininet).

Cách 1: bằng lệnh của mininet như sau:

Option 1:

B1: Lệnh để tuỳ chỉnh số Switch và số host connect của mỗi Switch.

Ví dụ: chay lệnh sau để có **4 switch** và **mỗi switch liên kết 2 host** với ip của máy controller là **192.168.58.129** .

```
sudo mn --controller=remote,ip=192.168.58.129 --topo=linear,4 --
switch=ovsk,protocols=OpenFlow13 --host=2
```

B2: Lệnh tạo topo mạng từ b1

```
sudo mn --controller=remote,ip=192.168.58.129 --topo=linear,4 --
switch=ovsk,protocols=OpenFlow13
```

Option2:

```
sudo mn --topo=tree,depth=2,fanout=3 --controller=remote --mac --switch ovsk --controller=remote,ip=192.168.58.129,port=6633
```

Trong đó:

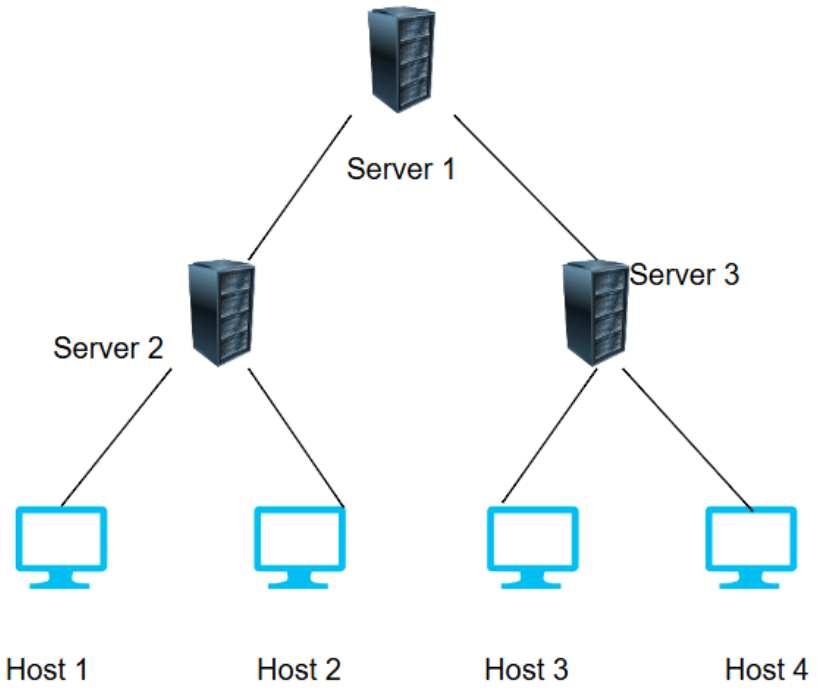
--topo=tree,depth=2,fanout=3: Tạo mạng dạng cây với 2 tầng và mỗi switch kết nối với 3 host.

--controller=remote: Sử dụng bộ điều khiển từ xa (OpenDaylight) trên IP 192.168.58.129 với cổng 6633.

--mac: Gán địa chỉ MAC cho các host và switch.

Cách 2: Chạy bằng file cript để tạo topo theo mô hình mong muốn

Bước 1: Chuẩn bị mô hình như sau



Hình 7 Ví dụ tạo topo mạng theo mong muốn

Theo mô hình ta thấy tổng 3switch(s) và 4host(h) với các link gồm:

(s1,s2) (s1,s3),(s2,h1) (s2,h2) (s3,h3) (s3,h4)

<u>Bước 2</u>: Tạo file cript topo3s4h.py theo mô hình trên

```
from mininet.topo import Topo


class MyTopo(Topo):
    "Simple topology example."


    def __init__(self):
        "Create custom topo."


        # Initialize topology
        Topo.__init__(self)


        # Add hosts
        h1 = self.addHost('h1')
        h2 = self.addHost('h2')
        h3 = self.addHost('h3')
        h4 = self.addHost('h4')


        # Add switches
        s1 = self.addSwitch('s1')
```

```
        s2 = self.addSwitch('s2')

        s3 = self.addSwitch('s3')


        # Add links between switches

        self.addLink(s1, s2)

        self.addLink(s1, s3)


        # Add links between switches and hosts

        self.addLink(s2, h1)

        self.addLink(s2, h2)

        self.addLink(s3, h3)

        self.addLink(s3, h4)


# Define topology to be used by Mininet

topos = {'tp': (lambda: MyTopo())}


# To run this topology, use the following command:

# sudo mn --controller=remote,ip=127.0.0.1,port=6653 --custom code2.py -
-topo tp
```

<u>Bước 3</u>: Chạy lệnh để tạo topo mạng

```
sudo mn --controller=remote,ip=192.168.58.129,port=6653 --custom topo3s4h.py --topo tp
```

Note:

+ ip là ip của controller và topo3s4h.py là file cript

+ Xem mô hình topo qua giao diện trên trình duyệt bằng cách:

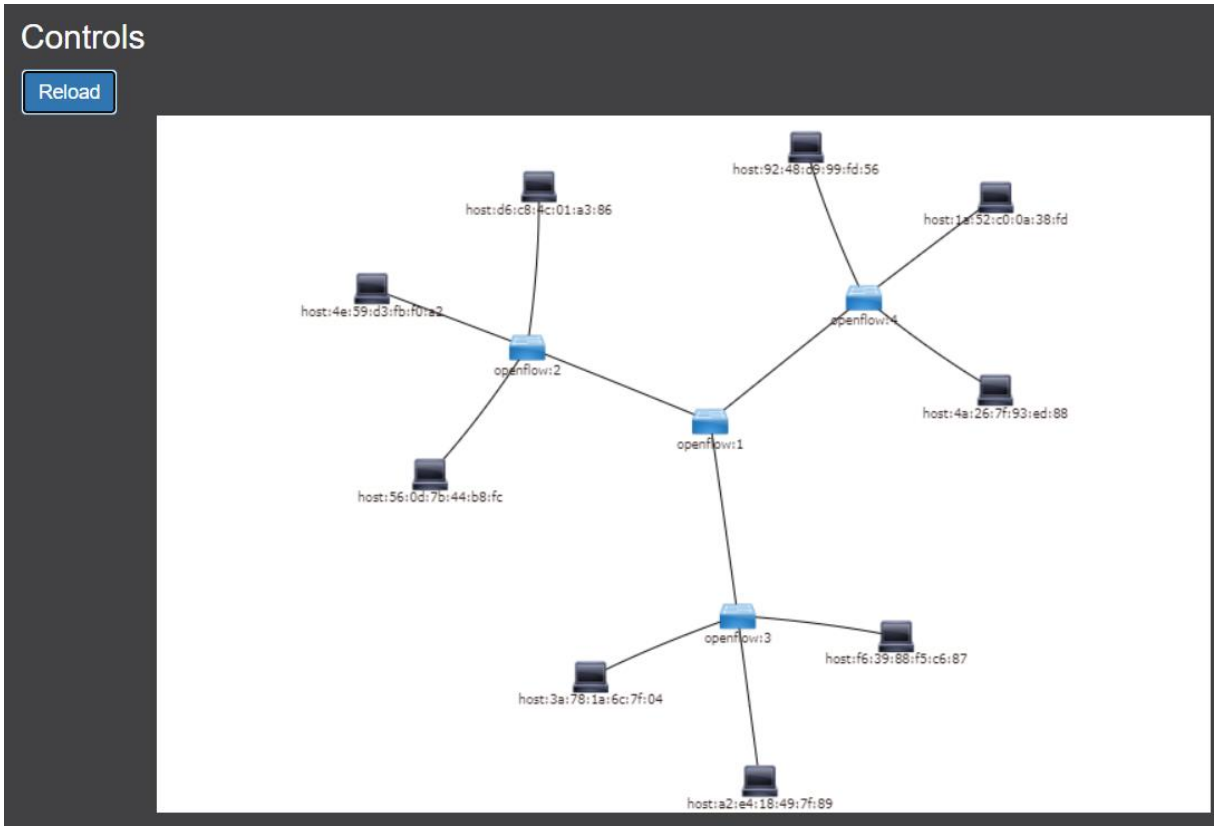
Dùng trình duyệt bất kì, nhập url như sau:

http<ipadd_controller>:8181/index.html>

Ví dụ máy controller ODL có ip: 192.168.58.129, ta nhập:

<u>http://192.168.58.129:8181/index.html</u>

+ Kiểm tra kết nối giữa các host bằng lệnh: pingall



Hình 8 Hình ảnh trước khi chưa custom name cho thiết bị

**Cài đặt Curl bên máy Ubunto (ODL)

```
Sudo apt update
sudo apt install curl
```

Lấy thông tin switch bằng Curl

```
curl -X GET http://192.168.58.129:8181/restconf/operational/opendaylight-inventory:nodes/ -u admin:admin -H "Content-Type: application/json"
```

Lấy thông tin host bằng Curl

```
curl -X GET \http://192.168.58.129:8181/restconf/operational/network-topology:network-topology/topology/flow:1/ \-u admin:admin \-H "Content-Type: application/json"
```

NOTE:

Bên máy mininet (image) - bản dòng lệnh

Lệnh cài xterm

```
sudo apt-get update
sudo apt-get install xterm
```

Lệnh cài wireshark, hping3, snort ở phía trên

Cách đo băng thông và độ trễ bằng Iperf: (so sánh trong trường hợp bình thường và lúc bị tấn công bằng hping3)

Bước 1: Ở máy mininet cài đặt Iperf

```
sudo apt-get install iperf3
```

Bước 2: Mở terminal của host h9 (host nạn nhân của Ddos và là Iperf server)

```
xterm h9
```

Bước 3: trong terminal của h9 chạy lệnh đặt h9 làm iperf server.

```
iperf3 -s
```

Chế độ **-s** sẽ đặt host **h9** làm server, lắng nghe các kết nối.

Bước 4: Khởi tạo một máy khách (client) iPerf trên host khác để kiểm tra hiệu suất

Trên các host khác như h1, bạn sẽ chạy iPerf trong chế độ client để kết nối tới server (host h9). Mở terminal cho host h1:

```
xterm h1
```

Sau đó chạy lệnh sau để gửi dữ liệu tới server (h9):

```
iperf3 -c 10.0.0.9
```

-c để đặt h1 làm client và kết nối tới địa chỉ IP của h9 (ở đây là 10.0.0.9).
Lúc này sẽ nhìn thấy thông số về **băng thông** và **độ trễ** giữa h1 và h9 trong điều kiện bình thường.

Bước 5: Mô phỏng tấn công DDoS với hping3

Tiếp theo, bạn có thể thực hiện tấn công DDoS từ host khác như h3 để gửi lưu lượng tấn công tới host h9.

Ví dụ, từ host h3, chạy lệnh sau để gửi tấn công ICMP flood:

```
hping3 -1 --flood 10.0.0.9
```

Do máy yếu sẽ bị đơ nên set lượng gói tin truyền thấp bằng cách thêm --count vào lệnh:

```
hping3 -1 --flood --count 200 10.0.0.9
```

B6: Đo hiệu suất trong quá trình tấn công

Trong khi các tấn công đang diễn ra,chạy lại iPerf từ 1 host khác (h1) để kiểm tra sự thay đổi hiệu suất mạng của h9:

```
iperf3 -c 10.0.0.9
```

Nhìn kết quả sẽ thấy sự suy giảm rõ rệt về băng thông và hiệu suất khi h9 bị tấn công DDoS.

Khi mạng đột nhiên bị giảm băng thông và tăng độ trễ tức có vấn đề, lúc này cần dùng wireshark để theo dõi và phân tích các gói tin trong mạng.

B6: Theo dõi tấn công bằng Wireshark

sử dụng **Wireshark** trên h9 để theo dõi các gói tin trong quá trình tấn công.

Note: các lệnh tuỳ chỉnh thêm để đo bằng iperf3:

Kiểm tra độ trễ (latency)

```
iperf3 -c <IP_Server> -u -b 1M
```

- -u: Chạy iPerf ở chế độ UDP.
- -b 1M: Tốc độ truyền tải mong muốn (ở đây là 1 Mbps). Bạn có thể điều chỉnh giá trị này.

Để kiểm tra băng thông trong một khoảng thời gian dài hơn (mặc định là 10 giây), bạn có thể sử dụng tùy chọn -t:

```
iperf3 -c <IP_Server> -t 60
```

Để kiểm tra nhiều kết nối song song, sử dụng tùy chọn -P:

```
iperf3 -c <IP_Server> -P 5
```

Để kiểm tra ở cả hai chiều (bi-directional test), sử dụng tùy chọn -R:

```
iperf3 -c <IP_Server> -R
```